Three Decades of Deception Techniques in Active Cyber Defense - Retrospect and Outlook

Li Zhang and Vrizlynn L. L. Thing

arXiv:2104.03594v1 [cs.CR] 8 Apr 2021

***Abstract*—Deception techniques have been widely seen as a game changer in cyber defense. In this paper, we review representative techniques in honeypots, honeytokens, and moving target defense, spanning from the late 1980s to the year 2021. Techniques from these three domains complement with each other and may be leveraged to build a holistic deception based defense. However, to the best of our knowledge, there has not been a work that provides a systematic retrospect of these three domains all together and investigates their integrated usage for orchestrated deceptions. Our paper aims to fill this gap. By utilizing a tailored cyber kill chain model which can reflect the current threat landscape and a four-layer deception stack, a two-dimensional taxonomy is developed, based on which the deception techniques are classified. The taxonomy literally answers which phases of a cyber attack campaign the techniques can disrupt and which layers of the deception stack they belong to. Cyber defenders may use the taxonomy as a reference to design an organized and comprehensive deception plan, or to prioritize deception efforts for a budget conscious solution. We also discuss two important points for achieving active and resilient cyber defense, namely deception in depth and deception lifecycle, where several notable proposals are illustrated. Finally, some outlooks on future research directions are presented, including dynamic integration of different deception techniques, quantified deception effects and deception operation cost, hardware-supported deception techniques, as well as techniques developed based on better understanding of the human element.**

***Index Terms*—Cyber defense, deception techniques, honeypots, honeytokens, moving target defense, computer network defense**

I. Introduction

In his book *The Art of Deception* [1], Kevin Mitnick, the world's most infamous hacker, asserted that the human element is security's weakest link. By attacking this link through various deception based social engineering techniques such as pretexting and phishing, cyber criminals have achieved wide success. For instance, according to 2019 Verizon data breach investigations report [2], phishing attacks accounted for more than 80% of reported security incidents. In the COVID-19 pandemic, we have also witnessed the enormous quantity of cases where hackers exploited coronavirus fears to deliver their phishing and malware attacks [3]–[5].

Deception aims to manipulate humans' perception by exploiting their psychological vulnerabilities [6], which has direct impact on their beliefs, decisions, and actions. It can be a powerful tool for both hackers and cyber defenders. In as early as the late 1980s, Clifford Stoll managed to set up an imaginary computer environment (now known as *honeypot*), in which a fictitious account was created along with a number of fake documents with enticing names, to lure a hacker to reveal himself and his objectives [7]. In the battle between the hacker and the cyber defender, a conventional wisdom is that the offense has the upper hand: cyber defenders have to make sure everything is properly maintained and prevent intrusions at every single point, whereas hackers may just need to take advantage of one vulnerability to breach the defense [8]. At the same time, attackers can always gain knowledge about a target system or network through a variety of reconnaissance and discovery tactics, while defenders are usually short of intelligence about their adversaries. Such asymmetric disadvantage for cyber defenders is well promised to be re-balanced through the use of defensive deception, which is expected to deliver a game-changing impact on how threats are faced [9]–[11].

The perimeter-based defense strategy utilizing conventional security measures such as firewalls, authentication controls, and intrusion prevention systems (IPS) has been proven feeble against infiltration. Even with the defense-in-depth strategy [12], where multiple layers of the conventional security controls are placed throughout the target network, cyber defenders still find it hard to prevent and detect sophisticated attacks like Advanced Persistent Threat (APT) based intrusions. Such targeted attacks typically exploit zero-day vulnerabilities to establish footholds on the target network and leave very few traces of their malicious activities behind for detection. Besides, conventional anomaly detection solutions such as intrusion detection systems (IDS) and behavior based malware scanners tend to raise an overwhelming number of false positive alerts, which plagues cyber defenders and hurts their efficacy in identifying and responding to the true attacks. Defensive deception, featured by its capability of detecting zero-day vulnerabilities and its low false alarm rates due to a clear line between legitimate user activities and malicious interactions, can act as an additional layer of defense to mitigate the issues.

Instead of focusing on attackers' actions, defensive deception works on their perception by obfuscating the attack surface. The objective is to hide critical assets from attackers and confuse or mislead them, thereby increasing their risk of being detected, causing them to misdirect or waste resources, delaying the effect of attacks, and exposing the adversary tradecraft prematurely [13]. In other words, defensive deception helps establish an *active cyber defense* posture, wherein the key elements are to anticipate attacks before they happen, to increase the costs of the adversary, and to gather new threat intelligence for preventing similar attacks.

The authors are with Cybersecurity Strategic Technology Center, ST Engineering, Singapore 609602. (email: zhang.li@stengg.com; vrizlynn.thing@stengg.com)

Since Stoll's honeypot, there have been numerous honeypots of different flavors proposed. These honeypots can be classified from different perspectives, such as whether they are server-based or client-based, of low interaction or high interaction, and based on real systems or virtual machines (VMs). Despite of the various flavors, all the honeypots share the same definition of being a security resource whose value lies in being probed, attacked, or compromised [14]. The term *honeypot* typically refers to decoy computer systems. Multiple interconnected honeypots form a honeynet. For bait resources that are of other forms (e.g., accounts, user files, database entries, and passwords), they can be collectively termed as *honeytokens* [15], [16]. Take the honeyfiles proposed in [17] as an example. These spurious files reside on a file server; once they are accessed, the server will send an alarm to alert a possible intrusion. Honeypots and honeytokens, when used in tandem, can introduce multi-tier fake attack surfaces for intruders. Unless the intruder can correctly select his target at every turn, his maneuver will be detected.

[redacted] th static deployment and configurations, the adversary will have enough time to infer their existence, map out them, and in turn evade them. Even worse, honeypots, especially the high-interaction ones which offer the intruder a real Operating System (OS) environment to interact with, may be exploited by the intruder to gain privileged control and used as a pivot point to compromise other systems [18]. This is where the [redacted] omes into the picture, which was identified as a key cybersecurity R&D theme by U.S. NITRD Program [19]. Specifically, [redacted] ck [redacted] with the attacker continuously confused and misled. For instance, Cohen reported in [21] that a combination of MTD techniques and honeytokens (e.g., automated responses on all unused ports), can help achieve long-term effectiveness of deceptions.

In this paper, we present a systematic review on the three aspects of defensive deception techniques (i.e., honeypots, honeytokens, and MTD) that have been proposed in the past three decades. The aim is to facilitate a better understanding of the advancement in each aspect and provide clues on how to better integrate them to build a holistic and resilient deception based defense. We limit our scope to techniques that can be directly applied to counter network intrusions. For example, the client-side honeypots [22], [23], which manifest as vulnerable user agents and actively troll malicious servers to study the client-side attacks, will be excluded in our survey [redacted] n view of this, the surveyed methods are classified based on the attack phases where they can be applied as countermeasures. In particular, we will use our proposed [redacted] which is specifically developed to model the network intrusion end to end and can reflect the current threat landscape. In each unique phase of the kill chain model, we further categorize the defensive deception methods according to [redacted]

MITRE

There have been some excellent surveys on cyber defensive deception. However, most of them just focus on either honeypots and honeytokens [24]–[28] or MTD techniques [29]–[33]. Although the survey of deception technology in [34] includes MTD techniques, its main focus is on honeypots and honeytokens; MTD techniques are just briefly mentioned and not systematically reviewed. In [20], twenty-four articles that use game theory to model defensive deception, comprising honeypots, honeytokens, and MTD techniques, are surveyed. Despite of representing an important direction, these game-theoretic models are just a small part of the literature. To the best of our knowledge, there has not been a work that provides a systematic retrospect of honeypots, honeytokens, as well as MTD techniques and investigates their integrated usage for orchestrated deceptions. Our survey aims to fill this gap.

The remainder of this paper is organized as follows. The proposed cyber kill chain model is illustrated in Section II, while the survey method is presented in Section III. Representative honeypots, honeytokens, and MTD techniques that can disrupt the adversary kill chain are reviewed in Section IV to VI, respectively. Section VII discusses how to use the deception techniques to achieve active and resilient cyber defense from two aspects, i.e., deception in depth and deception lifecycle. Finally, the paper is concluded in Section VIII with reflection and outlook of defensive deception research.

II. Cyber Kill Chain Model

A sophisticated cyber attack typically has to go through multiple consecutive phases before accomplishing its objective, be it stealthy collection and exfiltration of sensitive data or violation of critical assets' integrity or availability. The Lockheed Martin's intrusion kill chain [35] has been widely applied to assist structured analyses of the phased progressions. Nonetheless, [redacted] is often criticized for reinforcing the perimeter-focused thinking and failing to cover the attack paths inside the network perimeter [36], [37]. There have been several efforts aimed at expanding it for improved coverage. For instance, to explicitly model the intruder's movement from an initially compromised system to the target system, Laliberte [38] proposes to add a [redacted] Besides, Laliberte removes the *weaponization* phase as it happens outside of the victim network and no security measure can directly defend against it. The removal of the *weaponization* phase is in line with the kill chain model proposed in [39], which further introduces the *privilege escalation* and *exfiltration* phase. By contrast, the kill chain models proposed in [40], [41] significantly expands the Lockheed Martin's kill chain model. Both of them divide the

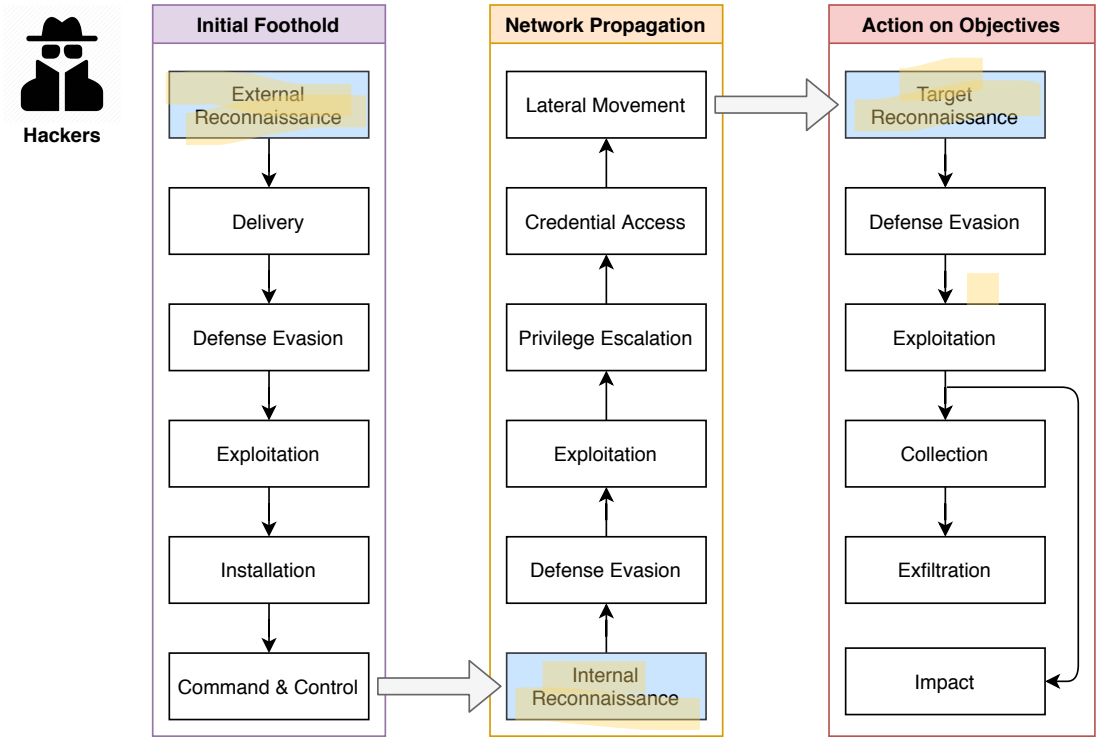


Fig. 1: The proposed cyber kill chain model

adversary kill chain into three sub-kill chains, i.e., the external kill chain aiming to establish an initial foothold, the internal kill chain aiming to propagate inside the victim network, and the target manipulation kill chain aiming to manipulate the target system to achieve attack objectives. In particular, the kill chain model in [41] includes four tactics from the MITRE ATT&CK model[1], namely *defense evasion*, *credential access*, *execution*, and *collection*, as additional attack phases.

Our proposed cyber kill chain model, shown in Figure 1, is developed based on [40], [41]. The purpose is to address some of their limitations. For example, there is only one attack objective in [40], represented by the final *execution* phase (i.e., activating malware to subvert operations of the target system). This omits many other possible impacts that a threat actor may incur. Besides, defense evasion dominated the attack tactics in 2019 [42], which we believe should be explicitly modeled in the kill chain. However, it is missing in [40]. Regarding the kill chain model in [41], we think its *weaponization* and *pivoting* phases are superfluous. For the former, we are in consonance with [38], [39] that it is not actionable to cyber defenders; for the latter, its actions and benefits actually have already been encompassed by the *lateral movement* phase. In addition, the MITRE ATT&CK model is recently supplemented with a new class of adversary tactics, i.e., *impact*. The inclusion of an *impact* phase in a kill chain model will enable it to model attack objectives more comprehensively.

In the proposed kill chain model, there are also three sub-kill chains, whose intents have been described above. Each modeled sub-kill chain starts with a *reconnaissance* phase as it provides the attacker with crucial information (such as network topology, vulnerabilities, and deployed security tools) to move further along the kill chain. All the three sub-kill chains also typically contain the *defense evasion* phase due to the widely adopted defense-in-depth strategy. No matter where the intruder propagates, he has to ensure that his maneuver is not detected by defensive measures. The *installation* phase in the external kill chain embodies both the *persistence* and *execution* tactics in the MITRE ATT&CK model. The internal kill chain for network propagation may be repeated for several times before the attacker finally reaches the target system. In the target manipulation kill chain, a combination of the *collection* and *exfiltration* phases is used to model the attacker's action of covertly stealing sensitive data, while the *impact* phase is used to represent the action of manipulating, interrupting, or destroying the critical assets.

With the intrusion activities covered end to end, the proposed kill chain model can be used to guide attack analyses, threat intelligence extraction, as well as defensive measures selection and prioritization. Such an intelligence-driven, threat-focused approach is essential to establish the active cyber defense posture against threats from both external actors and malicious insiders. For example, synthesis of the remaining kill chain of a detected attack may reveal a zero-day exploit [35]. This yields insights into possible future attacks and thereby drive the defender to implement countermeasures beforehand.

To facilitate the coordinated selection and deployment of deception techniques, the honeypot, honeytoken, and MTD techniques to be reviewed will be mapped to the proposed kill chain model's unique attack phases as listed in Table I. Based on the characteristics of each unique attack phase, the possible deception layers, in which the specific attack phase

[1]https://attack.mitre.org/

may be disrupted, are ticked accordingly.

III. Survey Method

To build the paper repository for this survey, we firstly searched in two leading research databases (i.e., IEEE Xplore and ACM Digital Library) with the keyword *cyber deception*, which returned a list of 253 research articles. Then we utilized the title and abstract of each paper to determine its relevance to our survey. The selection criteria is whether the paper is on defensive cyber deception (including evasion techniques of defensive deception). For example, papers on cyber deception attacks [43]–[46] were removed. This reduced the list to 87 research works. By including relevant papers cited in these works, we managed to collect additional articles. Together with some writings that we saw in other venues or mediums and think important to our survey (e.g., Ph.D. dissertations available online), the final repository contains 192 research works, which are referred to as primary studies in a systematic survey [47]. Although our paper repository does not cover all the related papers that were published in the past three decades, we are confident that most representative deception techniques have been included, through which the overall development trends in this field are accurately pictured.

IV. Honeypots

Honeypots can be broadly classified into two categories: research and production [48]. Although research honeypots play an important role in gathering intelligence on the threat landscape, they do not directly benefit a specific organization. In contrast, production honeypots are placed in an organization's environment for attack detection and risk mitigation. They may be deployed as sacrificial lamb, hacker zoo, minefield, proximity decoys, redirection shield, and deception ports (on production systems) [49], as described in Table II.

Stoll's honeypot [7] is a good example of the sacrificial lamb, which is the oldest and maybe also the most intuitive strategy. Being usually isolated from production systems, the sacrificial lamb honeypot may be easily identified and bypassed by attackers. The same limitation is shared by the hacker zoo. The minefield honeypots are commonly placed near the network perimeter, which will sound alarms upon attacker probing. This strategy helps enhance the perimeter based defense, but cannot handle attackers already inside the network. Both the proximity decoys and the redirection shield aim to lead the attacker astray and away from production systems. Their difference lies in that the redirection shield strategy, through the use of traffic rerouting or port redirection, does not require honeypots to be in the production network and hence has more flexibility. Among the five honeypot deployment strategies, the deception ports on production systems can be seen as the final defense. The various simulated vulnerable services on well-known ports can be used to detect and delay the attack even if the adversary reaches the production system. For example, the Deception Toolkit [50], which is the first open source honeypot, can set up the deception services.

Featured by deceiving to detect, derail and/or delay attacks, honeypots may be used to disrupt a number of attack phases in the cyber kill chain model, namely the reconnaissance, delivery, exploitation, installation, C2, lateral movement, and impact phase. On the other hand, in the defense evasion phase, the attacker may be able to identify honeypots and evade them. The remaining part of this section will be on these two aspects.

A. Disrupting the cyber kill chain

The intruder relies on successful reconnaissance to achieve tactical advantage in the campaign. Sticky honeypots can be used to mitigate the threat from network scans. For example, LaBrea [51] can take over unused IP addresses in the network and create virtual hosts to attract worms and hackers; connection attempts to the impersonated hosts will then be tarpitted. Greasy [52] further improves the sticky connection parameters to generate more realistic traffic. Besides slowing down the scanning activities, both LaBrea and Greasy are able to produce false network topologies and hence get adversaries confused. To dissimulate the network topology, many other honeypot techniques can also be used. For instance, HoneyD [53] can simulate a large number of virtual systems with configurable fingerprints and provide arbitrary services and routing topologies. These honeypot techniques are typically of low interaction and virtually adopt the minefield or proximity decoys deployment strategy.

To disrupt the attack phases such as delivery, C2, and lateral movement, the key is to direct the malicious traffic to high-interaction honeypots. By providing high-fidelity forged environment to interact with attackers, the exploitation, installation, and impact phases may also be broken. In addition, attackers' time and resources will be wasted and their tactics, techniques, and procedures (TTPs) may be revealed. As these high-interaction honeypots typically monitor one IP address each and have the problem of limited field of view, the redirection shield strategy is often adopted. In [54], it is proposed to handle anomalous traffic identified by IDS by a shadow honeypot, as shown in Figure 2. The shadow honeypot is an instrumented instance of the application (e.g., transactional applications) in protected system and share all internal states. Attacks mounted in the shadow honeypot will be caught and the induced state changes will be discarded, while legitimate traffic misclassified by IDS will be validated in the shadow honeypot and transparently handled. OpenFire [55] presents additional false targets by appearing to attackers that all IPs and ports of an organization network are open. Suspicious traffic will then be forwarded to a cluster of decoy machines. In the cloud environment, Biedermann *et al.* [56] propose to redirect potential attacks against an operational VM to a honeypot VM created through a live cloning process. The honeypot VM has exactly the same configuration as the original VM, but without the sensitive data. This way, the impact of the attack can be analyzed without risking the integrity of the original target VM. Similarly, in [57], the endpoint VM suspected of malicious activities will be cloned and forked in a deception environment with the same network and system configurations of the real network environment. If the suspicions for the VM are not found, the VM may be migrated back to the operational environment; otherwise, all

TABLE I: Unique attack phases in the proposed cyber kill chain model

Attack Phase	Description	Deception Layer			
		Network	System	Software	Data
Reconnaissance	Gather information from open-source intelligence, internet-facing/internal system probing, or network traffic sniffing	✓	✓	✓	✓
Delivery	Deliver the tailored malicious payload through entry vectors like email, websites, and removable media	✓	✓	✓	
Defense Evasion	Evade defense by disabling security tools, abusing trusted process to hide malware, obfuscation/encryption, etc.	✓	✓	✓	
Exploitation	Exploit identified vulnerabilities or simply user negligence to facilitate the following kill chain phases	✓	✓	✓	
Installation	Drop the payload (e.g., install a remote access Trojan or backdoor to maintain the foothold)		✓		
C2	Beacon outbound to establish a C2 channel with attacker for additional commands or payloads	✓	✓		
Privilege Escalation	Horizontally or vertically elevate privileges to gain access to sensitive assets		✓	✓	
Credential Access	Steal credentials like account names and passwords		✓	✓	✓
Lateral Movement	Move through the network by using installed remote access tools or stolen credentials	✓	✓	✓	✓
Collection	Collect data of interest from local databases/file system, screen/user input capture, shared network drives, etc.	✓	✓	✓	✓
Exfiltration	Process collected data and send it out through C2 channel or other channels	✓	✓		
Impact	Disrupt availability or compromise integrity of critical data or system/network services	✓	✓	✓	✓

TABLE II: Common Deployment Strategies for Honeypots [49]

Strategy	Description
Sacrificial Lamb	An isolated system that has no entry point to production systems
Hacker Zoo	An entire subnet of honeypots with varied platforms, services, vulnerabilities, and configurations, which are isolated from production systems
Minefield	A number of honeypots placed in forefront to serve as first attack targets
Proximity Decoys	Honeypots deployed in close proximity to production systems
Redirection Shield	External honeypots that appear on production systems through port redirection
Deception Ports	Simulated services (e.g., SMTP, DNS, FTP) on production systems

artifacts related to the attack in the deception environment can be documented for further scrutiny.

Besides standard IT systems, the above honeypot concepts are also applicable to industrial control systems (ICS). In [58], after analysing the threat landscape and unique security requirements of supervisory control and data acquisition (SCADA) systems, a plausible honeypot system is built, which is composed of both a low-interaction HoneyD honeypot emulating the programmable logic controller (PLC) and a high-interaction honeypot using a genuine PLC. In [59], HoneyD is extended to address the authenticity flaw of emulated PLCs; together with the proxy technology, multiple high-interaction honeypots can be distributed at the cost of a single actual PLC. A number of recent honeypot applications in ICS are based on Conpot [60], which is a low-interaction virtual ICS honeypot designed for easy deployment, modification and extension and supports a range of common industrial control protocols such as Modbus TCP, SNMP, and BACnet. In [61], a high-interaction honeypot is created by improving Conpot in the aspects of control protocol, human-machine interface (HMI) and equipment simulation. In [62], with the use of Conpot and the IMUNES network simulator, a complex high-interaction ICS is emulated.

B. Evading the honeypot

Despite of being a powerful tool to trap, delay, and even gather information about intruders, honeypots have their own weakness. At best, they are counterfeits of the real target. If intruders are able to identify honeypots, they will circumvent them or keep the malicious payload dormant, making honeypots useless. To some extent, attackers are highly motivated to push the detection of honeypots to early phases of their kill chain, so that their intrusion efforts are not rendered in vain and their TTPs are not disclosed.

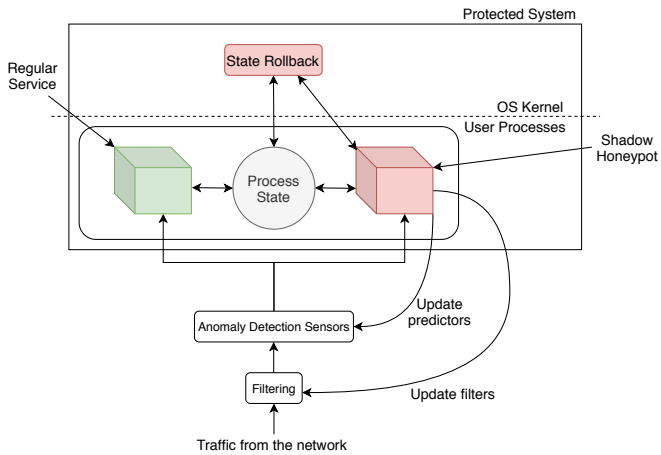


Fig. 2: The Shadow Honeypot architecture in [54]

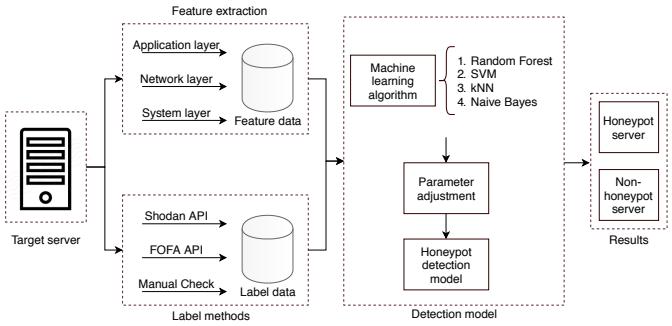


Fig. 3: The machine learning based Honeypot server identification method in [73]

Honeypots may be fingerprinted based on timing or behavior discrepancies in probing responses. After the introduction of the seminal HoneyD, it was soon found that it can be remotely fingerprinted based on its response to bad packets [63] or the latency of its emulated network links [64]. Degreaser in [65] can efficiently fingerprint sticky honeypots like LaBrea by sending a series of specially crafted probe packets; real hosts can then be discerned from tarpits based on the response. In [66], [67], by leveraging the flaw of many honeypots' reliance on off-the-shelf libraries to implement the transport layer, distinguishing probes constructed at this layer is able to systematically fingerprint honeypots.

Other unique features of honeypots may also be taken advantage of by attackers. Honeypot evader [68] exploits honeypots' innate characteristic of not initiating any network traffic and attacks only the hosts with obvious network activity. In [69], by exploiting the liability constraint that cyber defenders cannot allow their honeypots to participate in real attacks that could cause damage to other entities, an attacker can detect honeypots by checking whether his compromised machines can successfully send out unmodified malicious traffic. A more specific example of this concept is given in [70], where spammers can simply check if an open proxy relay is a honeypot based on whether emails can be sent to themselves.

Besides exploiting a single factor to tell whether a target is honeypot, information collected from different factors may be combined to reach a more accurate decision. In [71], such combination is performed with Dempster-Shafer theory [72], while in [73], machine learning (ML) techniques are used. For the latter, the design is depicted in Figure 3.

V. Honeytokens

Honeytokens share the same concept of honeypots, whose value lies in being used illicitly. In fact, the history of honeytokens is as long as that of honeypots. Besides the fictitious files with tempting names and contents in Stoll's honeypot in the late 1980s [7], Spafford built files with Unix sparse file structure in 1990s [74], which are of small size on disk but will result in "endless" transfer for attacker's copy attempt.

Honeytokens have remarkable flexibility. They can be in the form of any digital entity and placed anywhere across an organization's environment. The polymorphism and omnipresence bring two benefits for the defense: even though attackers are able to evade some forms of honeytokens, they may still be trapped by others; the uncertainty of whether and where honeytokens are placed will slow down attackers and may even turn them away (i.e., the deterrent effect). Typically, the honeytoken is simple to deploy and cost effective, making it considered as an exciting new dimension for honeypot [16].

Unlike honeypots which usually can only disrupt specific attack phases in the kill chain from the network and system layer, the various forms of honeytokens can be applied to thwart almost all the attack phases through all the four layers of the deception stack.

In the external reconnaissance phase, before engaging the target network, attackers will actively gather information from open-source intelligence. For example, Project Spacecrab creates credential honeytokens in the form of Amazon Web Services (AWS) keys, and found that the average time for a hacker to exploit the honeytoken is just thirty minutes after it is posted on GitHub [75]. There is a wealth of personal information on social network platforms, from which attackers might find valuable tips to drive targeted phishing campaigns. If bogus profiles are disseminated on these platforms [76], attackers may be misdirected in the delivery attack phase. To increase the authenticity, Virvilis *et al.* [77] suggest that the created fake personas should have positions of interest to attackers, connections with people from both inside and outside the organization, valid email addresses, as well as real, but closely monitored, organization accounts. To facilitate the creation of the bogus profiles, a method for automatically generating realistic personally identifiable information (PII) based honeytokens is proposed in [78].

The internet-facing web servers of an organization is another important intelligence source in external reconnaissance. To confuse and misdirect malicious website visitors, decoy hyperlinks embedded in web pages are used in [79]. These hyperlinks are invisible to legitimate human users, but can be detected by automated programs. An algorithm is also proposed for optimal placement of the decoys in website pages. Brewer *et al.* [80] propose to add the decoy links under two design principles: the multiple-link principle where

multiple decoy links are positioned off the visible page and valid links remain in their original; the shadow-link principle where multiple, invisible decoy links are stacked at the same coordinates as the valid link. In [77], three types of honey tokens are proposed for public web servers: fake entries in `robots.txt` files (used to tell crawlers which web pages to crawl and which ones not to), invisible decoy links as described above (e.g., white links with white font), and fake credentials in HTML comments.

When attackers probe the target network for more information, deceptive responses can be utilized to confuse them and delay their progress. To conceal the operating system (OS) related information that may be retrieved by attackers via OS fingerprinting, host-based OS obfuscation is suggested in [81] as a deception technique. With the attacker being unsure of the OS or even assuming the wrong OS, his penetration will be impeded. In [82], the defender controls the fake routes to be presented to attackers who use traceroute to map the target network's topology. Instead of directly rejecting the connection after an attack is suspected, which either is a false positive or will inform the adversary of being detected, deceptive delays are suggested in [83], [84]. The defender can use excuses (e.g., a computation requires a long time) to keep the suspect waiting, and use the time to collect more evidence or reorganize the defense. Katsinis and Kumar [85], [86] propose to deploy honeytokens such as fake form fields, fake parameters, and fake files in the web server. Alarms from these honeytokens will be sent to a deception module, which is responsible for redirecting the attacker traffic to a honeypot and supplying the attacker with misinformation that his attack is successful. By leveraging the modular design of Apache web server, the deception module can be conveniently inserted between the metadata processor and the content generator, as shown in Figure 4. A similar framework for achieving deceptive response is proposed in [87], where the deception module is deployed as a transparent reverse proxy.

A vital part of the attacker kill chain is to bypass the defense in the target network. Taking advantage of attackers' fear of having their TTPs exposed and resources wasted, Rowe *et al.* [88] propose to plant clues in systems such that they appear as honeypots (i.e., fake honeypots) and thereby turn attackers away. The planted clues can be names of known honeypot tools, non-standard system calls in security critical subroutines, reduced number of common files, and appearance of the system being little used. Besides fake honeypots, the deception effects on attackers of "fake fake honeypots", which refer to real honeypots that pretend to be noticeable fake honeypots, are also investigated.

Exploitation is another imperative phase in the attacker kill chain. Only after successfully exploiting some vulnerabilities can the adversary gain escalated privilege and be able to move further in the kill chain. In [89], "booby trap" codes are inserted into the protected software or system during compilation or program loading. These booby traps remains dormant under normal operation but may be triggered by attackers' exploitation attempts. Once triggered, the booby trap can perform advanced forensics to identify the attack in real time and send attackers deceptive responses. Frederico

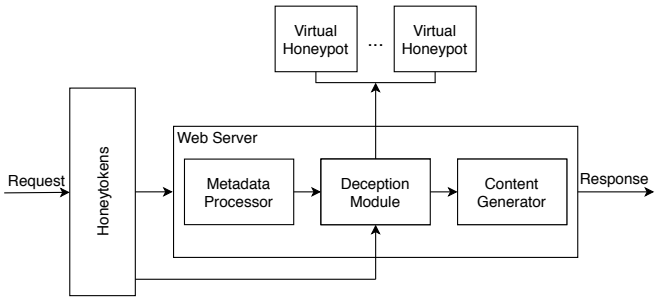


Fig. 4: The deceptive web server in Apache architecture in [86]

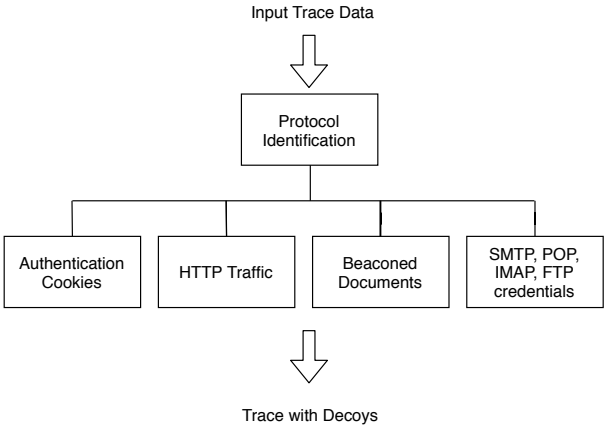


Fig. 5: The "record, modify, and replay" process for decoy traffic generation in [91]

et al. [90] propose to use decoy vulnerabilities that have been patched as honeytokens (aka honeypatches). In particular, the vulnerabilities are patched in such a way that attackers' exploitation attempts appear successful but their connections are actually redirected to an ephemeral honeypot with the unpatched version of the system or software. Besides, the honeypot may host a deceptive file system laced with disinformation to further deceive, delay, and misdirect attackers.

Attackers already inside the network will eavesdrop on the traffic to collect sensitive information and/or use the information to guide their following activities. For example, an attacker may map out systems that do not initiate any network traffic, which are likely to be honeypots, and circumvent them during the lateral movement. Such activity-guided target selection can be disabled by introducing decoy network and user space activities [68]. Bowen *et al.* [91], [92] propose to inject decoy traffic with enticing information that will induce the eavesdropper to take observable actions (e.g., using sniffed credentials to access a decoy account). In particular, to maximize the realism of the decoy traffic, a "record, modify, and replay" method (see Figure 5) is used to automatically generate a large amount of decoy traffic; the decoy traffic is also continuously updated to prevent an adversary from recognizing the bait over time. On the other hand, encryption may be used to restrict access to sensitive information in the network traffic. However, the eavesdropper may still reveal the secret through offline brute-force attacks. As decryption with a wrong key will result in random gibberish, the adversary will

know that he is successful if the output complies with some expected structure. To mitigate this risk, honey encryption (HE) [93] can be used. When the ciphertext generated by HE is decrypted by an incorrect key, a plausible-looking but bogus plaintext will be yielded. The adversary will be confused and may be misdirected to reveal himself if the bogus plaintext is a credential honeytoken. To make the bogus plaintext in HE more deceptive, i.e., contextually correct and domain specific, natural language processing (NLP) based techniques [94] and deep learning (DL) based ones [95] have been used.

In [96], [97], decoy permissions are used to extend role-based access control (RBAC) model for detecting the insider threat. These decoy permissions are not required for the specific roles to handle their tasks, and they are designed to give access to fake versions of sensitive assets. By monitoring attempts to access the fake assets, malicious users can be traced. We think that the decoy permissions are also useful for trapping outside attackers who have managed to infiltrate and reach the credential access attack phase. Legitimate users may know that they are not supposed to use the decoy permissions, but attackers who steal their credentials are not aware of that, leading to their activities being detected.

The following three categories of honeytoken techniques can be used to disrupt the last three attack phases in the kill chain model, namely the collection, exfiltration, and impact phase. As a result, threat actors may be hampered from achieving their objectives and their malicious activities may be detected.

Decoy passwords: Juels and Rivest [98] propose to assign multiple false passwords (aka honeywords) along with the real password to each account. This way, even though the adversary manages to crack the passwords from the stolen password hash files, he is still not sure which passwords are real. If the honeywords are used for login, an alarm will be set off. Instead of using multiple fake passwords to protect an account, Almeshekah *et al.* [99] propose to use a machine-dependent function (e.g., a physically unclonable function (PUF) [100] or a hardware security module (HSM) [101]) at the password server to generate "ersatzpasswords" from the stored password hashes; the hash of the ersatzpasswords are then stored in place of the original password hashes. This way, without physical access to the target's machine, any offline password cracking attempt will fail. If the attacker is unaware of the scheme and use the recovered ersatzpassword to login, the system administrator will be alerted.

Decoy database entries: Decoy database objects like `TABLE CREDIT_CARDS` or `VIEW EMPLOYEES_SALARY` can be inserted into databases to lure attackers. Čenys *et al.* [102] propose to implement modules for Oracle database management system (DBMS), which are responsible for monitoring access to the honeytokens, alerting the DBMS administrator, and logging malicious activities. To address the challenge of creating realistic decoy entries, HoneyGen [103] extrapolates rules describing the data structure, attributes, constraints and logic of real data items, and then automatically generates artificial items that comply with these rules. Padayachee [104] proposes to leverage aspect-oriented programming (AOP) to seamlessly augment a target DBMS with the basic honeytoken deployment processes, namely honeytoken generation, distribution, management, and detection.

Decoy user/system files: Yuill *et al.* [17] propose to use a honeyfile system to generate and monitor baits files; once these files are accessed, alerts will be sent to the system user. To ensure the detectability, Bowen *et al.* [105] propose to embed multiple signals in the decoy files, including a unique watermark that can be detected when the file is loaded in memory or appears in network traffic, a beacon that will signal a remote web site once the file is opened, and bait information such as credential honeytokens that will trigger alerts once used. To maximize the likelihood of an attacker taking the bait (i.e., conspicuousness), Voris *et al.* [106], [107] propose some automated deployment methods which can strategically place the decoy files. With the aim of increasing the enticingness of the decoys, NLP techniques are used in [108], where the fake file content is generated based on substitution and transposition of words collected from the target directory and file system. Existing file-based deception techniques mainly focus on decoy user data files, while PhantomFS [109] proposes to use decoy system files. To prevent false alarms triggered by legitimate activities accessing the decoy system files, a hidden interface is introduced, through which the decoy files are excluded. As an attack has to invoke some system files, this approach can further improve the detection of the adversary, especially for disrupting the impact attack phase.

VI. Moving Target Defense

Sun Tzu once wrote "just as water remains no constant shape, in warfare there are no constant conditions" [110]. Similarly, in cyber defense, a dynamic, constantly evolving attack surface for the protected network is extremely valuable to retain a resilient security posture. MTD techniques seek to randomize network components to reduce the likelihood of a successful attack, increase network dynamics to reduce the lifetime of an attack, and diversify otherwise homogeneous systems to limit the damage of a large-scale attack [111]. In other words, MTD intensifies uncertainty and workload for attackers by making the protected network less static, less deterministic, and less homogeneous.

Similar to honeytokens, the various MTD techniques are able to disrupt the adversary kill chain through all the four layers of the deception stack.

In the external reconnaissance phase, attackers have to gain necessary knowledge about the target network before they can move on along the kill chain. This attack phase can be guarded against by obfuscating the following two aspects of network properties:

IP obfuscation: To prevent attackers from tracing hosts in the target network based on IP addresses, a number of techniques have been proposed. Two early examples are *dynamic network address translation* (DyNAT) [112] which is a protocol-obfuscation technique that can scramble source and destination IP addresses in packet headers and *network address space randomization* (NASR) [113] which modifies a DHCP server to have short IP address leases so that host machines' IP addresses are changed frequently. Many recent

techniques follow the line of randomly changing IP addresses. OpenFlow Random Host Mutation (OF-RHM) [114] is able to mutate IP addresses with high unpredictability and rate. In particular, OF-RHM frequently assigns each host a random virtual IP (vIP) address, which will be automatically translated to/from the real IP (rIP) address of the host at the network edge. As a result, IP mutation is transparent to host machines and will not disrupt any active connection. To manage the random host mutation efficiently and minimizes the operational overhead, software-defined networking (SDN) [115] is utilized, where a centralized approach is realized based on OpenFlow [116]. A variant of the method, called Random Host Mutation (RHM), is proposed in [117], which changes vIP addresses in a distributed fashion and can be deployed on traditional networks. Due to the IPv4 network's limited unoccupied address space, which reduces the unpredictability of IP address hopping, Dunlop *et al.* propose MT6D [118], [119]. By leveraging the immense address space of IPv6, MT6D makes it harder for attackers to locate and subsequently target host machines. Besides, by encapsulating the original packet in a tunnel, MT6D also allows to change the IP address at any time without disrupting ongoing sessions.

OS obfuscation: To defend against OS fingerprinting attacks, Kampanakis *et al.* [120] propose an SDN based method, which hides the OS information in the response to detected illicit traffic by randomizing TCP sequence numbers and payload patterns in TCP, UDP, and ICMP protocols. Zhao *et al.* [121] propose to further model the interaction between the fingerprinting attack and defense as a signaling game [122] and develop optimal fingerprint hopping strategies by analyzing the equilibriums of the game. A strategy selection algorithm is also proposed to maximize the defense utility.

The defense evasion phase may be disrupted by dynamically and continuously changing the placement of IDS over time. By creating uncertainty about the location of IDS, the likelihood of attackers' actions being detected will be increased. Venkatesan *et al.* [123] analyze the problem of deploying IDS across the network in a resource-constrained environment using a graph-theoretic approach and propose several deployment strategies based on centrality measures [124] that capture important properties of the network. Sengupta *et al.* [125] model the same problem as a two-player general-sum Stackelberg game [126]. Two scalable algorithms are designed to find the equilibrium of the game, which corresponds to optimal strategies for switching IDS placement that balance the overall security and usability. On the other hand, as many IDS have been based on artificial intelligence (AI) techniques [127]–[129], there have been several adversarial attacks against the underlying AI models to induce misclassification [130]–[132]. The AI models may also adopt the moving target concept to improve the resilience against adversarial attacks, e.g., by randomizing the classification schemes [133], [134] as depicted in Figure 6.

The exploitation attack phase may be guarded against by various dynamic system and software techniques, which are also helpful to disrupt the impact attack phase:

Dynamic System: Among others, the most commonly used technique for increasing system dynamics is address space

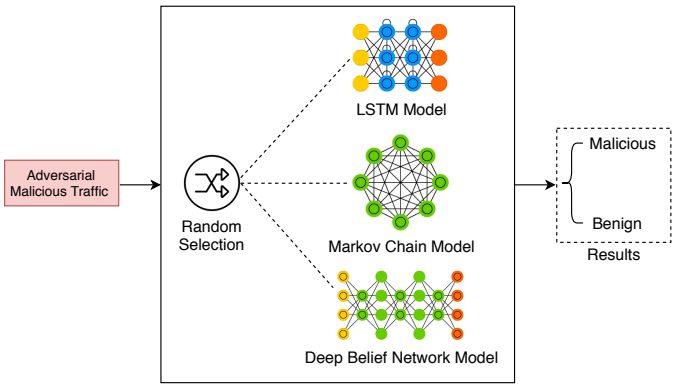


Fig. 6: Randomized classifiers to mitigate adversarial attacks, where malicious traffic perturbed to evade the long short term memory (LSTM) model will be detected by other models

layout randomization (ASLR) [135], [136], which hinders the exploitation of memory corruption vulnerabilities by randomizing memory addresses of a loaded software. To address code-injection attacks, an instruction set randomization (ISR) technique is proposed in [137], where an encoded version of software instructions is loaded into the memory and will be decoded by a key before being executed. Attackers' exploitation usually depend on vulnerabilities or characteristics of specific OS or CPU architectures. Thompson *et al.* [138] propose to enhance the security through a rotation of multiple OSs. Specifically, the method consists of several VMs equipped with different OSs. These VM hosts store shared data in a database and at one time only one of them will be mapped to an external IP address. The periodic rotation of VM hosts is controlled from an administrator machine running a daemon process, and the VM host that was previously in use is analyzed for evidence of intrusion and will be removed from rotation if compromised. Okhravi *et al.* [139] propose a TALENT framework to improve cyber survivability through platform diversity (i.e., different OSs and architectures). In TALENT, as depicted in Figure 7, a running application can be migrated between VMs with different platforms while preserving the state (e.g., the execution state, open files and network connections). A portable checkpoint compiler is used to facilitate the application live migration process. Note that the migration among different platforms must take less time than the time needed for attacking a specific platform. Or else, the migration actually diminishes security because threat actors now have a choice of multiple platforms to attack [111].

Dynamic Software: There is also a wide range of attacks exploiting software vulnerabilities, which requires precise understanding of the target software. By randomizing the implementation, software diversity introduces uncertainty in the target, increases the cost to attackers, and may provide an effective counter to side-channel attacks [140]. Chameleon-Soft [141] proposes to divide a complex software program into smaller tasks, each of which has a set of executable variants that are functionally equivalent but with different quality attributes (e.g., performance, robustness, and mobility). The executable variants can then be shuffled to change the

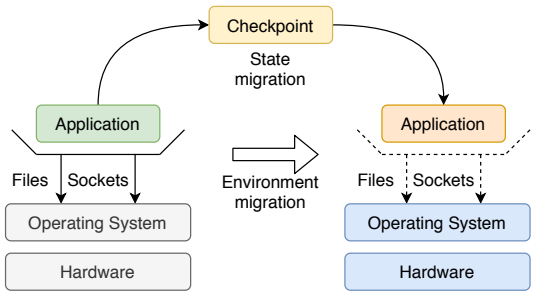


Fig. 7: The TALENT migration process in [139]

attack surface in accordance with different security situations. To defend against code reuse attacks, such as return-oriented programming (ROP), Gupta *et al.* [142] propose a fine-grained software diversity approach called Marlin. Marlin breaks a software binary into function blocks and randomly shuffles the order. Such a process can be performed transparently at load time, which ensures every execution instance of the software to be unique. On the other hand, to prevent a software program from being exploited by identified vulnerabilities, Le Goues *et al.* [143] propose an automatic software repair method called GenProg. By utilizing an extended form of genetic programming, GenProg is able to evolve a software program with identified vulnerabilities to a functionally equivalent variant that are no longer susceptible to the previous risks. The dynamically patched software can be legacy programs without formal specifications and annotations.

To prevent attackers who are already inside the network from eavesdropping on communication flows, Germano da Silva *et al.* [144] propose a multipath routing strategy, which relies on SDN features to frequently modify communication routes between SCADA devices. As each route transmits only a portion of the packets exchanged during the communication, even though the eavesdropper is well positioned in a strategic point of the network, he will not be able to intercept an entire communication between two devices. As the multipath routing strategy always relies on the shortest path to transmit the acknowledgment (ACK) packets from the receiver, Aseeri *et al.* [145] found that an attacker can still capture all the packets by eavesdropping on the shortest path and blocking the ACK packet corresponding to the packet sent through other routes until it is retransmitted via the path he is listening to. To address this defect, the SDN controller can be utilized to instruct the receiver to send the ACK packet via the path used by the sender. In the self-shielding dynamic network architecture (SDNA) [146], [147], packets go through one or more intermediate devices before reaching the receiver. The intermediate devices are not simply routers; they also rewrite traffic to conceal the sender and receiver's identities. As a result, the eavesdropping attack can also be thwarted.

The collection phase in the kill chain may be disrupted by dynamic data approaches. To prevent the cryptographic keys stored in the cloud from being extracted by attackers using cross-VM side-channel attacks [148], Pattuk *et al.* [149] propose to partition the keys into random shares based on the secret sharing and threshold cryptography [150]. The random shares are then stored in different VMs and will be regenerated periodically. As a result, the adversary has to attack multiple VMs to steal the key and the impact of a successful attack will be limited to a certain time period. On the other hand, dynamic data approaches may also impede the impact phase. Smutz and Stavrou [151] propose to randomize the data block order of Microsoft office documents while keeping the visual interpretation intact. As malicious payloads embedded in the documents usually rely on a specific order of internal components, the randomization prevents them from being executed.

VII. Deception for Active Defense

A. Deception in depth

Although each of the deception techniques surveyed in Section IV to Section VI is able to disrupt one or several kill chain phases, when used alone, attackers can always find a way to circumvent it. One example is the various honeypot evasion techniques described in Section IV-B. By contrast, when multiple deception techniques that complement with each other are used together, forming an overall deception fabric covering several or even all layers in the deception stack, a more resilient cyber defense posture can be established. It is believed that such a deception in depth strategy should be leveraged by organizations to achieve comprehensive defense against the onslaught of advanced adversaries and attack techniques [9]. In fact, there have already been commercial products implementing this strategy to create a complete illusion for the adversary [152].

A number of works have investigated the hybrid use of deception techniques. Through the combination, the deception effect on the adversary can be magnified, leading to the threat actor being deterred, delayed, distracted or detected.

Wang *et al.* [153] propose a multi-layer deception system (see Figure 8), which is composed of honeypot servers and various honeytokens such as honey people, honey files, honey database, and honey activities. The honey people is fake personas created on social network platforms. The honey activities are coupled with honey files and honeypot servers to prevent sophisticated attackers from discerning the bogus resources by observing user behaviors or network traffic. The alerts from all the deception entities are sent to the analyst server, where analysis is performed to confirm or remove the alerts. The analyst server may also correlate different alerts to extract more information of a penetration attempt. For example, if an alert is triggered on a honey file and later on a honey database entry, some correlation analyses may reveal that the two separate alerts correspond to the same espionage campaign.

A decoy-enhanced network address randomization method called DESIR (see Figure 9) is proposed in [154], which dynamically mutates the network topology with a number of decoy servers to invalidate attacker's knowledge about the network. DESIR consists of four main components, i.e., an authentication server, a randomization controller, a protected server pool, and a decoy bed. The authentication server is responsible for verifying the client's credential, providing

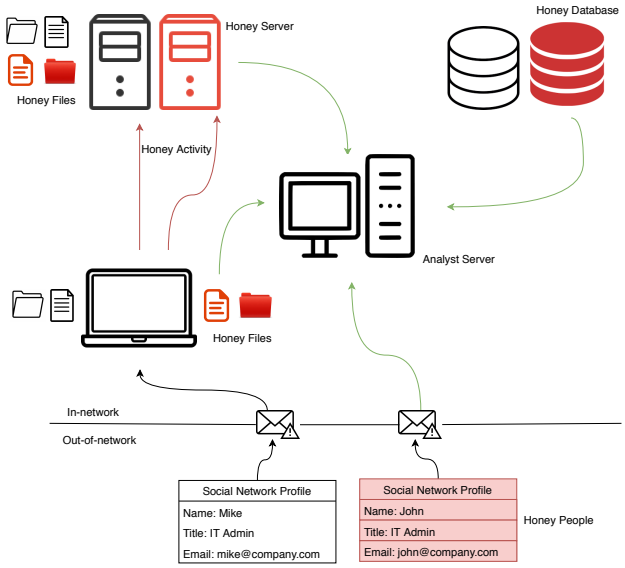


Fig. 8: The multi-layer deception system in [153]

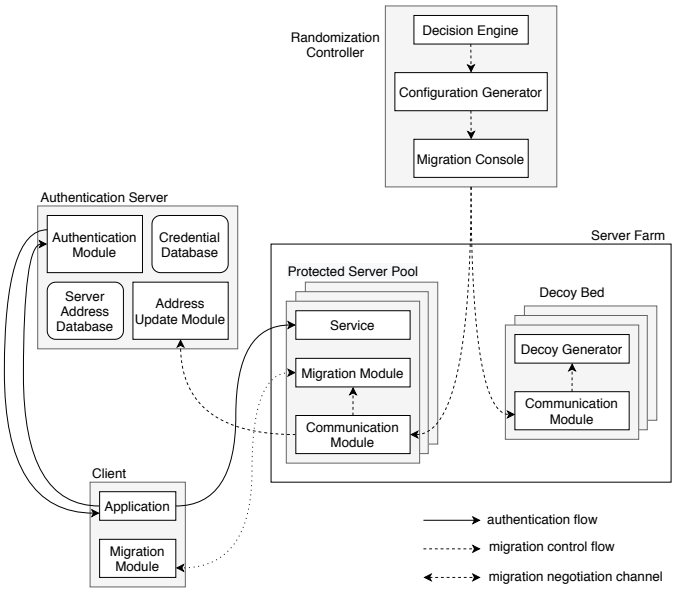
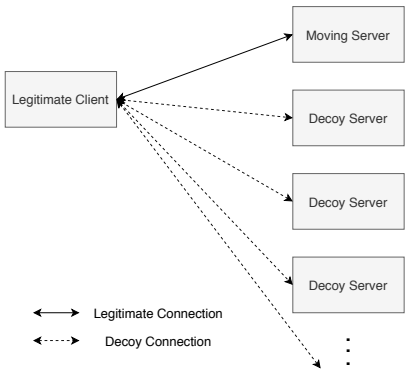


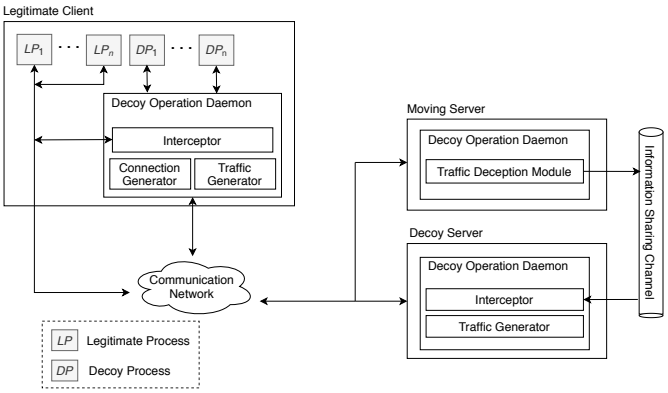
Fig. 9: The DESIR system in [154], which is based on decoy-enhanced network address randomization

requested server's current IP address upon successful authentication, and updating the IP addresses of servers in the server pool. The randomization controller coordinates the mutation of the network. Its decision module determines the frequency to randomize the network addresses, configuration generator module controls the overall topology of the network, and migration console module distributes the new configurations to the real and decoy servers. Upon receiving new configurations, the decoy generator in the decoy bed will update the decoy network, including the decoy server's IP addresses and MAC addresses as well as the installed or emulated OS and applications. The decoy bed may include both high-interaction and low-interaction honeypots as decoys, which depend on the received configurations. Although the honeypots can be used to attract attackers and learn their TTPs, their main function in DESIR is to further confuse attackers, prolong their network scanning time, and invalidate the knowledge that can be gained.

In the DESIR system, as shown in Figure 9, the authenticated client and the moving server is seamlessly connected via the migration module. However, this means that if the client is compromised, it will be easy for the adversary to trace the moving server by analyzing the network traffic. To cope with this threat, Park *et al.* [155] propose to inject decoy connection and traffic with the honeypot servers, as illustrated in Figure 10. To generate decoy traffic that is even convincing for sophisticated attackers, a context-aware traffic generation mechanism is used (see Figure 10b). On the client, the connection generator module of the decoy operation daemon is responsible for creating decoy connections with honeypot servers, and the traffic generator module creates decoy traffic of a similar pattern to the legitimate traffic. The traffic deception module on the moving server shares the characteristics of the outbound traffic, which are imitated by the traffic generator module on the decoy server to generate similar traffic with decoy processes on the client. Besides,



(a) Decoy connections



(b) Context-aware decoy traffic injection

Fig. 10: The decoy connection and traffic injection in [155]

similar to [156], OS fingerprint mutation is also applied on all servers so that the attack surface is further obfuscated.

The SDN based CHAOS system (see Figure 11) in [157] obfuscates the network attack surface by using honeypot (i.e., decoy servers), honeytoken (i.e, fake response to port scanning), and MTD (i.e, random host mutation) techniques.

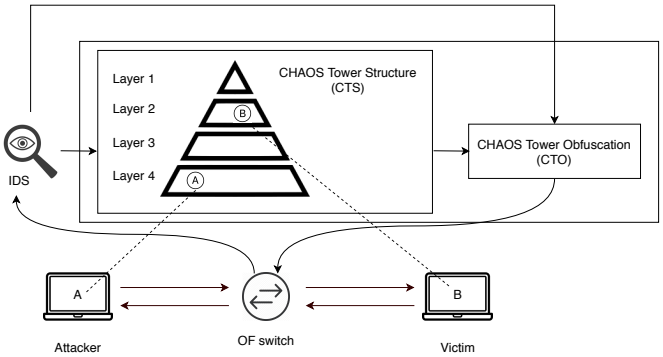


Fig. 11: The CHAOS system in [157], where suspicious connections identified by IDS or CTS are obfuscated by CTO

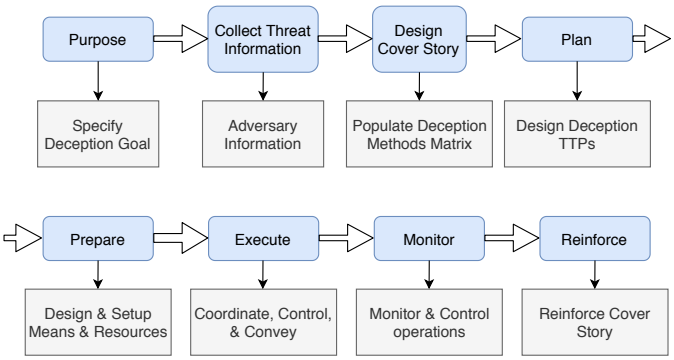


Fig. 12: The eight-phase cyber deception chain in [172]

In particular, host machines in the network is divided into several layers according to their security levels, which forms a CHAOS tower structure (CTS). Communication rules are defined in a CTS module. For example, connection requests from a host machine in lower layers to hosts in higher layers will be deemed as suspicious. The suspicious communications determined by the CTS module and other traffic identified by IDS as malicious will be forwarded to a CHAOS tower obfuscation (CTO) module, where the three types of techniques listed above are implemented. The three obfuscation strategies are applied based on a threshold factor, which can be controlled by the administrator according to the required security level and the structure of the protected network.

A complete list of the reviewed deception techniques are shown in Table III, where they are classified based on the two-dimensional taxonomy, i.e., which attack phases they can disrupt and which deception layer they belong to. Note that some methods, especially the hybrid ones, are able to disrupt multiple attack phases and use techniques from multiple layers. These methods are repeated in the table to fully indicate their characteristics and effects. The reconnaissance phase in Table III just refers to the external reconnaissance, while deception techniques that can guard against the internal reconnaissance and target reconnaissance attack phases are the same as those for the lateral movement phase. This separation is aimed to make it easier to understand the different applicabilities and effects of the numerous deception techniques for disrupting attacker reconnaissance.

B. Deception lifecycle

Deception techniques are employed to affect threat actors such that they take action or inaction to the advantage of cyber defenders. To achieve and maintain the desired perceptual and cognitive effects, deception mechanisms have to be properly designed and updated. Almeshekah [170] proposes a deception framework comprising three main phases, i.e., planning, implementing and integrating, and monitoring and evaluating. In the planning phase, the goal of deception is specified, the attacker's bias that can be exploited to achieve desired reactions is analyzed, and the risk that may be introduced by deception techniques is also assessed. De Faveri *et al.* [171] propose a goal-driven approach for designing the deception based defense. The approach integrates three phases, i.e., system modeling for specifying the goal, security modeling for specifying security concerns from the attacker perspective, and the deception modeling for specifying the defense (e.g., designing deception stories, monitoring channels, and deception metrics). The first two phases establish the context for modeling the deception. Heckman *et al.* [172] propose a deception chain for deception operation management from a lifecycle perspective, which is composed of eight phases as depicted in Figure 12. Note that its second phase to the fourth phase, i.e., collecting threat information, designing cover story, and planning, can be implemented by leveraging our two-dimensional taxonomy.

After the coordinated deception tactics are built and executed, they should evolve in response to environment changes and attacker's behavior [173]. Take the honeypot for example. A static honeypot is very likely to be detected by the adversary. By contrast, dynamic honeypots [174]–[176], besides their capability of learning about the network for automated deployment, can continuously monitor the network environment for changes and reconfigure themselves accordingly. Moreover, some dynamic honeypots are able to adapt based on their interactions with attackers. By taking advantage of reinforcement learning, Wagener *et al.* [177], [178] build honeypots that can learn to adopt the best behavior such as blocking or executing commands, returning erroneous messages, and insulting the adversary. The insults act as reverse Turing tests [179] and aim to identify whether the opponent is human or an automated tool. Based on the same concept, Pauna and Bica [180] build a self-adaptive honeypot that also emulates a secure shell (SSH) server, where an extra interaction strategy, i.e., delaying the command execution, is added.

Besides using reinforcement learning to improve the interaction of deception techniques with the adversary, game theory may also be utilized. Carroll and Grosu [181] model the interaction between the defender and the attacker as a signaling game, which is a non-cooperative two player dynamic game (i.e., the two players take turns to choose actions) of incomplete information. The incomplete information is due to the attacker's uncertainty of the target (e.g., whether the target system is a honeypot). Deceptive equilibrium strategies are then derived to achieve better defense of the network. Rahman *et al.* [182] model the interaction between OS fingerprinter

TABLE III: Deception techniques classified based on the two-dimensional taxonomy, where HP, HT, and MTD denotes honeypot, honeytoken, and moving target defense, respectively

Attack Phase	**Deception Layer**			
	Network	System	Software	Data
Reconnaissance	HP: [51]–[53], [55], [62], [154]–[158]	HP: [53], [58]–[61], [67], [159], [160]	HP: [53]	HP:
	HT: [82], [155], [157], [161]	HT: [21], [81], [83], [155], [161], [162]	HT: [77], [79], [80], [85]–[87]	HT: [75]–[78], [85], [86], [153], [163]
	MTD: [21], [112]–[114], [117]–[119], [154]–[157], [160], [164]–[166]	MTD: [120], [121], [155], [156]	MTD:	MTD:
Delivery	HP:	HP: [55]–[61], [67], [159]	HP: [54]	HP:
	HT:	HT:	HT:	HT:
	MTD: [113], [114], [117]–[119]	MTD:	MTD:	MTD:
Defense Evasion	HP: [64], [65]	HP:	HP:	HP:
	HT:	HT: [68], [88]	HT:	HT:
	MTD: [123], [125]	MTD: [133], [134]	MTD:	MTD:
Exploitation	HP: [77], [167]	HP: [55]–[61], [67], [153], [155], [156], [159], [160]	HP: [54]	HP:
	HT:	HT: [90], [109], [168]	HT: [77], [89], [90]	HT:
	MTD:	MTD: [135]–[139]	MTD: [141]–[143]	MTD:
Installation	HP:	HP: [55]–[61], [67]	HP:	HP:
	HT:	HT:	HT:	HT:
	MTD:	MTD:	MTD:	MTD:
C2	HP:	HP: [55]–[61], [67]	HP:	HP:
	HT:	HT:	HT:	HT:
	MTD:	MTD:	MTD:	MTD:
Privilege Escalation	HP:	HP:	HP:	HP:
	HT:	HT: [96], [97]	HT:	HT:
	MTD:	MTD:	MTD:	MTD:
Credential Access	HP:	HP:	HP:	HP:
	HT:	HT: [96], [97]	HT:	HT: [93]–[95], [98], [99], [153], [163]
	MTD:	MTD:	MTD:	MTD:
Lateral Movement	HP: [53], [55], [154]–[157]	HP: [53], [55]–[61], [67], [153], [160]	HP: [53], [54]	HP:
	HT: [68], [91], [92], [153], [155], [157], [161]	HT: [21], [68], [90], [153], [155], [161]	HT: [90]	HT: [98], [99], [153], [163]
	MTD: [21], [144]–[147], [154]–[157], [160], [164], [169]	MTD: [113], [114], [117]–[121], [155], [156]	MTD:	MTD:
Collection	HP:	HP: [55]–[61], [67]	HP: [54]	HP:
	HT:	HT: [17], [77], [96], [97], [153]	HT: [102], [104]	HT: [17], [93]–[95], [98], [99], [102]–[108], [153], [163]
	MTD: [144]–[147]	MTD:	MTD:	MTD: [149]
Exfiltration	HP: [77]	HP: [55]–[61], [67]	HP:	HP:
	HT:	HT:	HT:	HT:
	MTD:	MTD:	MTD:	MTD:
Impact	HP: [55]	HP: [55]–[61], [67], [153], [155], [156]	HP: [54]	HP:
	HT:	HT: [90], [109], [168]	HT: [89], [90]	HT:
	MTD:	MTD: [135]–[139]	MTD: [141]–[143]	MTD: [151]

and the defender as a signaling game and the equilibrium analysis results in a counter-fingerprinting mechanism called DeceiveGame. Unlike many other tools which alter all connections' outgoing packets to deceive fingerprinting and incur significant performance degradation, DeceiveGame can distinguish fingerprinters from benign clients and selectively mystify packets to confuse the fingerprinters, hence minimizing the side effects. Carter *et al.* [183] model the interaction as a two-palyer Stackelberg game to discover optimal moving target strategies (instead of simple randomization) for dynamic platforms based defense, while Lei *et al.* [184] model the confrontation in MTD as a Markov game to identify the optimal hopping strategy. In general, game theory makes it possible for cyber defenders to investigate how the adversary's belief evolves and influences his actions, and provides a quantitative framework for optimizing the manipulation of this belief to the benefit of defense [185].

In deception defense, it is critical to continuously monitor the feedback channels to decide whether the desired effects on attackers are achieved. Honeypots may be easily identified, evaded, and even compromised by the adversary, honeytokens may not be enticing, and the hopping frequency in MTD may not be high enough. If the feedback indicates the deception defense is lack of effectiveness, the deception strategies, tactics, and techniques must be immediately adjusted. For this part, game theory may also be helpful. For instance, by building a multi-layer game model, a feedback learning framework is developed in [186], which enables the system to monitor its current state and update the defense strategy based on the risk it estimates on the fly.

VIII. Reflection and Outlook

In cyber defense, deception techniques exploit attackers' psychological biases and vulnerabilities and have direct impact on their beliefs, decisions, and actions. Even just some clues that the target system's response may be fake will delay or even turn away the adversary [187]. Compared to conventional attack prevention or detection tools which can only impede the adversary's current actions, deception techniques may have long-term impact on the adversary. Nonetheless, as deception techniques typically involve active adversary engagement, they have to be carefully maintained to stay effective. Especially when addressing APT and insider threats, high-fidelity deception over a long period is necessary. This poses stringent requirements to the deception operator. Although a vast number of deception techniques in various domains have been proposed since their inception in late 1980s, very few of them achieve real-life applications. According to Lance Spitzner, deception techniques were held back not by the concept, but by the technology [188]. For example, early honeypots require manual customization and management, which is extremely time-consuming and error-prone. Only after recent advancement in virtualization and SDN technology, which simplifies and automates the tedious process, honeypot techniques become scalable in real-life networks. To further enhance the usability of deception techniques, some recent works [189], [190] propose to provide *deception as a service* through automatically orchestrated deception deployment with minimal human involvement. These efforts will definitely facilitate wide adoption of deception techniques. On the other hand, most of the early deception techniques have the drawback of assuming static network configurations, while recent dynamic techniques leveraging game theory models usually oversimplify the adversary's strategies [191]. These limitations make the actual deployment less effective and easy to be evaded by the adversary. We think recent efforts in testbeds and experimentation platforms [192], [193] is promising to solve this problem. With deception techniques tested and validated on realistic systems and in realistic settings, not only the possible design flaws can be identified much more easily, but also the effectiveness of different techniques can be compared for easier tradeoff or complementary usage. It has been shown both analytically and experimentally that a single deception technique is not enough to attain highly resilient cyber deception [194]. The testbed and experimentation platforms will be an ideal environment for finding the optimal composition of different deception building blocks.

The two-dimensional taxonomy, built based on our proposed cyber kill chain model and the four-layer deception stack, facilitates the systematic review of representative approaches from the domains of honeypots, honeytokens, and MTD techniques in a threat-focused manner. To create a holistic deception fabric covering the protected network and form a complete illusion for the adversary, an integrated use of these techniques is believed to be a prerequisite. Our taxonomy may serve as a guide or reference to consolidate and coordinate the different techniques. By adopting the deception in depth strategy and properly managing deception mechanisms throughout their lifecycle, a resilient deception defense will be built, which helps organizations establish the active cyber defense posture.

For future research directions, we think that there will be more works on effective integration of the deception techniques from different domains. A well-designed deception defense should fully exploit the characteristics of different deception techniques. As these characteristics often complement with each other, the overall deception effect may be magnified and the defense cost may be optimized. For instance, defenders may distribute low-cost honeytokens all over the network to monitor the security status. Based on the indicated threat level, the instances of high-interaction honeypots, the hopping frequencies of MTD techniques, and the density of decoy activities can be dynamically adjusted. Such context awareness will be further enhanced when deception defense is combined with conventional threat detection and response (TDR) solutions. On the other hand, to smooth the integration, the deception effects on the adversary and the cost of deception operations should be quantified. In fact, there have been some works in this direction. For the former, Maleki *et al.* [195] propose a Markov model based framework for analyzing MTD techniques, where security capacity is defined to measure their strength or effectiveness. For the latter, Wang *et al.* [153] model the design of the multi-layer deception system as an optimization problem to minimize the total expected loss due to system deployment and asset compromise. To better address these two problems, we feel that the quantitative framework

offered by game theory will play an important role.

We may also witness hardware become a more important participant in cyber defense. A lesson from the Spectre and Meltdown attacks [196] is that no security is possible if the underlying hardware is vulnerable. Conversely, a more secure hardware may better obfuscate the attack surface and boost the uncertainty. For instance, the Morpheus secure architecture in [197] implements a hardware based churning mechanism to transparently randomize key program values, which are needed by attackers for crafting successful attacks, at runtime. To enhance the value of the churning mechanism, Morpheus also incorporates an attack detector. Once sensing a potential attack, the detector can immediately trigger an increased churn rate to strengthen the defense and repel the attack. Besides, the ensembles of MTD techniques developed on Morpheus, such as relocating pointers and encrypting code and pointers, can use the hardware support to achieve more randomness at a lower cost.

The ultimate target of deception defense is the adversary's perception and belief. We think that there will also be more works developed based on better understanding of the human element. Ferguson-Walter [198] suggests that advances in behavioral science should be leveraged to better influence attacker's target selection and operations. By manipulating threat actors' cognitive biases and cognitive load, it will be made more difficult for them to achieve their objectives.

REFERENCES

[1] K. D. Mitnick, W. L. Simon, and S. Wozniak, *The Art of Deception: Controlling the Human Element of Security*, 1st ed. Wiley, Aug. 2007.

[2] "2019 Data Breach Investigations Report," https://enterprise.verizon.com/resources/reports/2019-data-breach-investigations-report.pdf.

[3] C. Cimpanu, "Czech hospital hit by cyberattack while in the midst of a COVID-19 outbreak," https://www.zdnet.com/article/czech-hospital-hit-by-cyber-attack-while-in-the-midst-of-a-covid-19-outbreak/, Mar. 2020.

[4] D. Palmer, "Coronavirus-themed phishing attacks and hacking campaigns are on the rise," https://www.zdnet.com/article/coronavirus-themed-phishing-attacks-and-hacking-campaigns-are-on-the-rise/, Mar. 2020.

[5] C. Cimpanu, "There's now COVID-19 malware that will wipe your PC and rewrite your MBR," https://www.zdnet.com/article/theres-now-covid-19-malware-that-will-wipe-your-pc-and-rewrite-your-mbr/, Apr. 2020.

[6] G. Cybenko, A. Giani, and P. Thompson, "Cognitive hacking: A battle for the mind," *Computer*, vol. 35, no. 8, pp. 50–56, Aug. 2002.

[7] C. Stoll, *The Cuckoo's Egg: Tracking a Spy through the Maze of Computer Espionage*. USA: Doubleday, 1989.

[8] W. J. Lynn III, "Defending a New Domain: The Pentagon's Cyberstrategy," *Foreign Affairs*, vol. 89, no. 5, pp. 97–108, 2010.

[9] Lawrence Pingree, "Emerging Technology Analysis: Deception Techniques and Technologies Create Security Technology Business Opportunities," Gartner, Inc., Tech. Rep., Jul. 2015.

[10] K. Ferguson-Walter, S. Fugate, J. Mauger, and M. Major, "Game theory for adaptive defensive cyber deception," in *Proceedings of the 6th Annual Symposium on Hot Topics in the Science of Security - HotSoS '19*. Nashville, Tennessee: ACM Press, 2019, pp. 1–8.

[11] T. Shade, A. Rogers, K. Ferguson-Walter, S. B. Elsen, D. Fayette, and K. Heckman, "The Moonraker Study: An Experimental Evaluation of Host-Based Deception," in *Hawaii International Conference on System Sciences*, 2020.

[12] U.S. Department of Homeland Security, "Recommended Practice: Improving Industrial Control System Cybersecurity with Defense-in-Depth Strategies," Sep. 2016.

[13] R. Ross, V. Pillitteri, R. Graubart, D. Bodeau, and R. McQuaid, "Developing cyber resilient systems: A systems security engineering approach," National Institute of Standards and Technology, Gaithersburg, MD, Tech. Rep. NIST SP 800-160v2, Nov. 2019.

[14] L. Spitzner, *Honeypots: Tracking Hackers*. Boston: Addison-Wesley Professional, Sep. 2002.

[15] Augusto Paes de Barros, "IDS: RES: Protocol Anomaly Detection IDS - Honeypots," https://seclists.org/focus-ids/2003/Feb/95, Feb. 2003.

[16] Lance Spitzner, "Honeytokens: The Other Honeypot," Security Focus information, Jul. 2003.

[17] J. Yuill, M. Zappe, D. Denning, and F. Feer, "Honeyfiles: Deceptive files for intrusion detection," *Proceedings from the Fifth Annual IEEE SMC Information Assurance Workshop*, 2004.

[18] Lance Spitzner, "Problems and Challenges with Honeypots," Security Focus information, Jan. 2004.

[19] "NITRD CSIA IWG Cybersecurity Game-Change Research & Development Recommendations," 2010.

[20] J. Pawlick, E. Colbert, and Q. Zhu, "A Game-Theoretic Taxonomy and Survey of Defensive Deception for Cybersecurity and Privacy," *arXiv:1712.05441 [cs]*, May 2019.

[21] Fred Cohen, "Moving target defenses with and without cover deception," http://all.net/Analyst/2010-10.pdf, 2010.

[22] C. Seifert, I. Welch, and P. Komisarczuk, "HoneyC - The Low-Interaction Client Honeypot," in *Proceedings of the 2007 NZCSRCS*, Jan. 2007.

[23] J. Nazario, "PhoneyC: A Virtual Client Honeypot," in *USENIX Workshop on Large-Scale Exploits and Emergent Threats*, 2009.

[24] Christian Seifert, Ian Welch, and Peter Komisarczuk, "Taxonomy of Honeypots," Tech. Rep. CS-TR-06-12, Jun. 2006.

[25] M. Nawrocki, M. Wählisch, T. C. Schmidt, C. Keil, and J. Schönfelder, "A Survey on Honeypot Software and Data Analysis," *arXiv:1608.06249 [cs]*, Aug. 2016.

[26] Scott Smith, "Catching Flies: A guide to the various flavors of honeypots," SANS, Tech. Rep., 2016.

[27] X. Han, N. Kheir, and D. Balzarotti, "Deception Techniques in Computer Security: A Research Perspective," *ACM Computing Surveys*, vol. 51, no. 4, Jul. 2018.

[28] A. M. Efendi, Z. Ibrahim, M. A. Zawawi, F. Abdul Rahim, N. M. Pahri, and A. Ismail, "A Survey on Deception Techniques for Securing Web Application," in *2019 IEEE 5th Intl Conference on Big Data Security on Cloud (BigDataSecurity), IEEE Intl Conference on High Performance and Smart Computing, (HPSC) and IEEE Intl Conference on Intelligent Data and Security (IDS)*, May 2019, pp. 328–331.

[29] H. Okhravi, M. A. Rabe, T. J. Mayberry, W. G. Leonard, T. R. Hobson, D. Bigelow, and W. W. Streilein, "Survey of Cyber Moving Target Techniques," Lincoln Lab, MIT, Tech. Rep. 1166, Sep. 2013.

[30] G.-l. Cai, B.-s. Wang, W. Hu, and T.-z. Wang, "Moving target defense: State of the art and characteristics," *Frontiers of Information Technology & Electronic Engineering*, vol. 17, no. 11, pp. 1122–1153, Nov. 2016.

[31] B.C. Ward, S.R. Gomez, R.W. Skowyra, D. Bigelow, J.N. Martin, J.W. Landry, and H. Okhravi, "Survey of Cyber Moving Targets: 2nd Edition," Lincoln Lab, MIT, Tech. Rep. 1228, Jan. 2018.

[32] C. Lei, H.-Q. Zhang, J.-L. Tan, Y.-C. Zhang, and X.-H. Liu, "Moving Target Defense Techniques: A Survey," *Security and Communication Networks*, vol. 2018, 2018.

[33] S. Sengupta, A. Chowdhary, A. Sabur, A. Alshamrani, D. Huang, and S. Kambhampati, "A Survey of Moving Target Defenses for Network Security," *arXiv:1905.00964 [cs]*, Mar. 2020.

[34] D. Fraunholz, S. D. Anton, C. Lipps, D. Reti, D. Krohmer, F. Pohl, M. Tammen, and H. D. Schotten, "Demystifying Deception Technology:A Survey," *arXiv:1804.06196 [cs]*, Apr. 2018.

[35] E. M. Hutchins, M. J. Cloppert, and R. M. Amin, "Intelligence-Driven Computer Network Defense Informed by Analysis of Adversary Campaigns and Intrusion Kill Chains," *Leading Issues in Information Warfare & Security Research*, vol. 1, no. 1, 2011.

[36] P. Reidy, "Combating the Insider Threat at the FBI: Real World Lessons Learned," in *Black Hat*, USA, 2013.

[37] Giora Engel, "Deconstructing The Cyber Kill Chain," https://www.darkreading.com/attacks-breaches/deconstructing-the-cyber-kill-chain/a/d-id/1317542, Nov. 2014.

[38] Marc Laliberte, "A Twist On The Cyber Kill Chain: Defending Against A JavaScript Malware Attack," https://www.darkreading.com/attacks-breaches/a-twist-on-the-cyber-kill-chain-defending-against-a-javascript-malware-attack/a/d-id/1326952, Sep. 2016.

[39] Blake D. Bryant and Hossein Saiedian, "A novel kill-chain framework for remote security log analysis with SIEM software," *Computers & Security*, vol. 67, pp. 198–210, Jun. 2017.

[40] S. T. Malone, "Using an Expanded Cyber Kill Chain Model to increase attack resiliency," in *Black Hat*, USA, 2016.

[41] P. Pols, "The Unified Kill Chain," Ph.D. dissertation, Cyber Security Academy, Dec. 2017.

[42] Kelly Sheridan, "Defense Evasion Dominated 2019 Attack Tactics," https://www.darkreading.com/vulnerabilities—threats/defense-evasion-dominated-2019-attack-tactics/d/d-id/1337457, Mar. 2020.

[43] S. Amin, X. Litrico, S. Sastry, and A. M. Bayen, "Cyber Security of Water SCADA Systems—Part I: Analysis and Experimentation of Stealthy Deception Attacks," *IEEE Transactions on Control Systems Technology*, vol. 21, no. 5, pp. 1963–1970, Sep. 2013.

[44] N. Hou, Z. Wang, D. W. C. Ho, and H. Dong, "Robust Partial-Nodes-Based State Estimation for Complex Networks Under Deception Attacks," *IEEE Transactions on Cybernetics*, vol. 50, no. 6, pp. 2793–2802, Jun. 2020.

[45] Q. Zhang, K. Liu, Y. Xia, and A. Ma, "Optimal Stealthy Deception Attack Against Cyber-Physical Systems," *IEEE Transactions on Cybernetics*, vol. 50, no. 9, pp. 3963–3972, Sep. 2020.

[46] R. Meira-Goes, S. Lafortune, and H. Marchand, "Synthesis of Supervisors Robust Against Sensor Deception Attacks," *IEEE Transactions on Automatic Control*, 2021.

[47] B. Kitchenham, "Procedures for Performing Systematic Reviews," *Keele, UK, Keele University*, p. 33, 2004.

[48] Lance Spitzner, "The Value of Honeypots, Part One: Definitions and Values of Honeypots," Security Focus information, Oct. 2001.

[49] B. Scottberg, W. Yurcik, and D. Doss, "Internet honeypots: Protection or entrapment?" in *IEEE International Symposium on Technology and Society (ISTAS'02)*. Raleigh, NC, USA: IEEE, 2002, pp. 387–391.

[50] Fred Cohen, "Deception ToolKit," http://www.all.net/dtk/, Mar. 1998.

[51] Tom Liston, "LaBrea: "Sticky" Honeypot and IDS," http://labrea.sourceforge.net/labrea-info.html, 2001.

[52] Leslie Shing, "An improved tarpit for network deception," Ph.D. dissertation, Naval Postgraduate School, 2016.

[53] N. Provos, "A virtual honeypot framework," in *Proceedings of the 13th Conference on USENIX Security Symposium - Volume 13*, ser. SSYM'04. San Diego, CA: USENIX Association, Aug. 2004, p. 1.

[54] K. G. Anagnostakis, S. Sidiroglou, P. Akritidis, K. Xinidis, E. Markatos, and A. D. Keromytis, "Detecting Targeted Attacks Using Shadow Honeypots," in *USENIX Security*, Jan. 2005, p. 16.

[55] K. Borders, L. Falk, and A. Prakash, "OpenFire: Using deception to reduce network attacks," in *2007 Third International Conference on Security and Privacy in Communications Networks and the Workshops - SecureComm 2007*, Sep. 2007, pp. 224–233.

[56] S. Biedermann, M. Mink, and S. Katzenbeisser, "Fast dynamic extracted honeypots in cloud computing," in *Proceedings of the 2012 ACM Workshop on Cloud Computing Security Workshop*. Raleigh, North Carolina, USA: Association for Computing Machinery, Oct. 2012, pp. 13–18.

[57] V. E. Urias, W. M. S. Stout, and H. W. Lin, "Gathering threat intelligence through computer network deception," in *2016 IEEE Symposium on Technologies for Homeland Security (HST)*. Waltham, MA, USA: IEEE, May 2016, pp. 1–6.

[58] J. P. Disso, K. Jones, and S. Bailey, "A Plausible Solution to SCADA Security Honeypot Systems," in *2013 Eighth International Conference on Broadband and Wireless Computing, Communication and Applications*, Oct. 2013, pp. 443–448.

[59] M. Winn, M. Rice, S. Dunlap, J. Lopez, and B. Mullins, "Constructing cost-effective and targetable industrial control system honeypots for production networks," *International Journal of Critical Infrastructure Protection*, vol. 10, pp. 47–58, Sep. 2015.

[60] Lukas Rist, Johnny Vestergaard, Daniel Haslinger, Andrea Pasquale, and John Smith, "Conpot," http://conpot.org/, 2015.

[61] C. Zhao and S. Qin, "A research for high interactive honepot based on industrial service," in *2017 3rd IEEE International Conference on Computer and Communications (ICCC)*. Chengdu: IEEE, Dec. 2017, pp. 2935–2939.

[62] S. Kuman, S. Groš, and M. Mikuc, "An experiment in using IMUNES and Conpot to emulate honeypot control networks," in *2017 40th International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO)*, May 2017, pp. 1262–1268.

[63] Joseph Corey, "Advanced Honey Pot Identification and Exploitation," Phrack Inc., Tech. Rep., 2003.

[64] X. Fu, W. Yu, D. Cheng, X. Tan, K. Streff, and S. Graham, "On Recognizing Virtual Honeypots and Countermeasures," in *2006 2nd IEEE International Symposium on Dependable, Autonomic and Secure Computing*, Sep. 2006, pp. 211–218.

[65] L. Alt, R. Beverly, and A. Dainotti, "Uncovering network tarpits with degreaser," in *Proceedings of the 30th Annual Computer Security Applications Conference on - ACSAC '14*. New Orleans, Louisiana: ACM Press, 2014, pp. 156–165.

[66] A. Vetterl and R. Clayton, "Bitter harvest: Systematically fingerprinting low- and medium-interaction honeypots at internet scale," in *Proceedings of the 12th USENIX Conference on Offensive Technologies*, ser. WOOT'18, Baltimore, MD, USA, Aug. 2018.

[67] A. M. Vetterl, "Honeypots in the age of universal attacks and the Internet of Things," Ph.D. dissertation, University of Cambridge, Nov. 2019.

[68] J. L. Rrushi, "Honeypot Evader: Activity-guided Propagation versus Counter-evasion via Decoy OS Activity," in *Proceedings of the 14th IEEE International Conference on Malicious and Unwanted Software*, Nantucket, Massachusetts, USA, Oct. 2019, p. 11.

[69] P. Wang, L. Wu, R. Cunningham, and C. C. Zou, "Honeypot detection in advanced botnet attacks," *International Journal of Information and Computer Security*, vol. 4, no. 1, p. 30, 2010.

[70] N. Krawetz, "Anti-honeypot technology," *IEEE Security Privacy*, vol. 2, no. 1, pp. 76–79, Jan. 2004.

[71] O. Hayatle, A. Youssef, and H. Otrok, "Dempster-Shafer Evidence Combining for (Anti)-Honeypot Technologies," *Information Security Journal: A Global Perspective*, vol. 21, no. 6, pp. 306–316, Jan. 2012.

[72] K. Sentz, S. Ferson, and K. Sentz, "Combination of evidence in Dempster-Shafer theory," Sandia National Laboratories, Albuquerque, New Mexico., Tech. Rep., 2002.

[73] C. Huang, J. Han, X. Zhang, and J. Liu, "Automatic Identification of Honeypot Server Using Machine Learning Techniques," *Security and Communication Networks*, vol. 2019, pp. 1–8, Sep. 2019.

[74] Eugene H. Spafford, "More than passive defense," https://www.cerias.purdue.edu/, Jul. 2011.

[75] D. Bourke and D. Grzelak, "Breach Detection at Scale with AWS Honey Tokens," in *Black Hat Asia*, 2018.

[76] G. Stringhini, C. Kruegel, and G. Vigna, "Detecting Spammers on Social Networks," in *Proceedings of the 26th Annual Computer Security Applications Conference*, 2010.

[77] N. Virvilis, B. Vanautgaerden, and O. S. Serrano, "Changing the game: The art of deceiving sophisticated attackers," in *2014 6th International Conference On Cyber Conflict (CyCon 2014)*. Tallinn, Estonia: IEEE, Jun. 2014, pp. 87–97.

[78] J. White, "Creating Personally Identifiable Honeytokens," in *Innovations and Advances in Computer Sciences and Engineering*, T. Sobh, Ed. Dordrecht: Springer Netherlands, 2010, pp. 227–232.

[79] D. Gavrilis, I. Chatzis, and E. Dermatas, "Flash Crowd Detection Using Decoy Hyperlinks," in *2007 IEEE International Conference on Networking, Sensing and Control*, Apr. 2007, pp. 466–470.

[80] Douglas Brewer, Kang Li, Laksmish Ramaswamy, and Calton Pu, "A Link Obfuscation Service to Detect Webbots," Jul. 2010.

[81] S. B. Murphy, J. T. McDonald, and R. F. Mills, "An Application of Deception in Cyberspace: Operating System Obfuscation," in *International Conference on Information Warfare and Security*, Apr. 2010.

[82] S. T. Trassare, "A technique for presenting a deceptive dynamic network topology," Ph.D. dissertation, Naval Postgraduate School, 2013.

[83] D. P. Julian, "Delaying-type responses for use by software decoys," Ph.D. dissertation, 2002.

[84] Neil C. Rowe, "Deception in defense of computer systems from cyber-attack," in *Cyber War and Cyber Terrorism*, A. Colarik and L. Janczewski, Eds., 2007.

[85] C. Katsinis and B. Kumar, "A Security Mechanism for Web Servers Based on Deception," in *Proceedings on the International Conference on Internet Computing (ICOMP)*, 2012, p. 6.

[86] ——, "A Framework for Intrusion Deception on Web Servers," in *International Conference on Internet Computing(ICOMP)*, 2013, p. 7.

[87] X. Han, N. Kheir, and D. Balzarotti, "Evaluation of Deception-Based Web Attacks Detection," in *Proceedings of the 2017 Workshop on Moving Target Defense - MTD '17*. Dallas, Texas, USA: ACM Press, 2017, pp. 65–73.

[88] N. C. Rowe, E. J. Custy, and B. T. Duong, "Defending Cyberspace with Fake Honeypots," *Journal of Computers*, vol. 2, no. 2, pp. 25–36, Apr. 2007.

[89] S. Crane, P. Larsen, S. Brunthaler, and M. Franz, "Booby trapping software," in *Proceedings of the 2013 Workshop on New Security Paradigms Workshop - NSPW '13*. Banff, Alberta, Canada: ACM Press, 2013, pp. 95–106.

[90] F. Araujo, K. W. Hamlen, S. Biedermann, and S. Katzenbeisser, "From Patches to Honey-Patches: Lightweight Attacker Misdirection, Deception, and Disinformation," in *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '14. Scottsdale, Arizona, USA: Association for Computing Machinery, Nov. 2014, pp. 942–953.

[91] B. M. Bowen, V. P. Kemerlis, P. Prabhu, A. D. Keromytis, and S. J. Stolfo, "A system for generating and injecting indistinguishable network decoys," *Journal of Computer Security*, vol. 20, no. 2-3, pp. 199–221, Jun. 2012.

[92] ——, "Automating the injection of believable decoys to detect snooping," in *Proceedings of the Third ACM Conference on Wireless Network Security - WiSec '10*. Hoboken, New Jersey, USA: ACM Press, 2010, p. 81.

[93] A. Juels and T. Ristenpart, "Honey Encryption: Security Beyond the Brute-Force Bound," Tech. Rep. 155, 2014.

[94] E. O. Abiodun, A. Jantan, O. I. Abiodun, and H. Arshad, "Reinforcing the Security of Instant Messaging Systems Using an Enhanced Honey Encryption Scheme: The Case of WhatsApp," *Wireless Personal Communications*, Jan. 2020.

[95] A. E. Omolara, A. Jantan, O. Isaac Abiodun, K. Victoria Dada, H. Arshad, and E. Emmanuel, "A Deception Model Robust to Eavesdropping Over Communication for Social Network Systems," *IEEE Access*, vol. 7, pp. 100 881–100 898, 2019.

[96] A. Juels, "A bodyguard of lies: The use of honey objects in information security," in *Proceedings of the 19th ACM Symposium on Access Control Models and Technologies - SACMAT '14*. London, Ontario, Canada: ACM Press, 2014, pp. 1–4.

[97] P. Kaghazgaran and H. Takabi, "Toward an Insider Threat Detection Framework Using Honey Permissions," *Journal of Internet Services and Information Securuirty (JISIS)*, 2015.

[98] A. Juels and R. L. Rivest, "Honeywords: Making password-cracking detectable," in *Proceedings of the 2013 ACM SIGSAC Conference on Computer & Communications Security - CCS '13*. Berlin, Germany: ACM Press, 2013, pp. 145–160.

[99] M. H. Almeshekah, C. N. Gutierrez, M. J. Atallah, and E. H. Spafford, "ErsatzPasswords: Ending Password Cracking and Detecting Password Leakage," in *Proceedings of the 31st Annual Computer Security Applications Conference on - ACSAC 2015*. Los Angeles, CA, USA: ACM Press, 2015, pp. 311–320.

[100] C.-H. Chang, Y. Zheng, and L. Zhang, "A Retrospective and a Look Forward: Fifteen Years of Physical Unclonable Function Advancement," *IEEE Circuits and Systems Magazine*, vol. 17, no. 3, pp. 32–62, 2017.

[101] "PCI Hardware Security Module Security Requirements, Version 1.0," PCI Security Standards Council, Tech. Rep., Apr. 2009.

[102] Antanas Čenys, Darius Rainys, Lukas Radvilavičius, and Nikolaj Goranin, "Implementation of Honeytoken Module in DBMS Oracle 9ir2 Enterprise Edition for Internal Malicious Activity Detection," in *IEEE Computer Society's TC on Security and Privacy*, 2005.

[103] M. Bercovitch, M. Renford, L. Hasson, A. Shabtai, L. Rokach, and Yuval Elovici, "HoneyGen: An automated honeytokens generator," in *Proceedings of 2011 IEEE International Conference on Intelligence and Security Informatics*, Jul. 2011, pp. 131–136.

[104] K. Padayachee, "Aspectising honeytokens to contain the insider threat," *IET Information Security*, vol. 9, no. 4, pp. 240–247, Dec. 2014.

[105] B. M. Bowen, S. Hershkop, A. D. Keromytis, and S. J. Stolfo, "Baiting Inside Attackers Using Decoy Documents," in *International Conference on Security and Privacy in Communication Systems*, 2009.

[106] J. Voris, J. Jermyn, A. D. Keromytis, and S. J. Stolfo, "Bait and Snitch: Defending Computer Systems with Decoys," in *Proceedings of the Cyber Infrastructure Protection Conference, Strategic Studies Institute*, 2013, p. 25.

[107] J. Voris, J. Jermyn, N. Boggs, and S. Stolfo, "Fox in the trap: Thwarting masqueraders via automated decoy document deployment," in *Proceedings of the Eighth European Workshop on System Security - EuroSec '15*. Bordeaux, France: ACM Press, 2015, pp. 1–7.

[108] Ben Whitham, "Automating the Generation of Enticing Text Content for High-Interaction Honeyfiles." IEEE computer society, 2017.

[109] J. Lee, J. Choi, G. Lee, S.-W. Shim, and T. Kim, "PhantomFS: File-Based Deception Technology for Thwarting Malicious Users," *IEEE Access*, vol. 8, pp. 32 203–32 214, 2020.

[110] S. Tzu, *The Art Of War*, first thus edition ed. Filiquarian, Nov. 2007.

[111] H. Okhravi, T. Hobson, D. Bigelow, and W. Streilein, "Finding Focus in the Blur of Moving-Target Techniques," *IEEE Security & Privacy*, vol. 12, no. 2, pp. 16–26, Mar. 2014.

[112] D. Kewley, R. Fink, J. Lowry, and M. Dean, "Dynamic approaches to thwart adversary intelligence gathering," in *Proceedings DARPA Information Survivability Conference and Exposition II. DISCEX'01*, vol. 1, Jun. 2001, pp. 176–185 vol.1.

[113] S. Antonatos, P. Akritidis, E. P. Markatos, and K. G. Anagnostakis, "Defending against hitlist worms using network address space randomization," in *Proceedings of the 2005 ACM Workshop on Rapid Malcode*, ser. WORM '05. Fairfax, VA, USA: Association for Computing Machinery, Nov. 2005, pp. 30–40.

[114] J. H. Jafarian, E. Al-Shaer, and Q. Duan, "Openflow random host mutation: Transparent moving target defense using software defined networking," in *Proceedings of the First Workshop on Hot Topics in Software Defined Networks - HotSDN '12*. Helsinki, Finland: ACM Press, 2012, p. 127.

[115] D. Kreutz, F. M. V. Ramos, P. E. Veríssimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-Defined Networking: A Comprehensive Survey," *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14–76, Jan. 2015.

[116] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "OpenFlow: Enabling innovation in campus networks," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69–74, Mar. 2008.

[117] E. Al-Shaer, Q. Duan, and J. H. Jafarian, "Random Host Mutation for Moving Target Defense," in *Security and Privacy in Communication Networks*, A. D. Keromytis and R. Di Pietro, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, vol. 106, pp. 310–327.

[118] M. Dunlop, S. Groat, W. Urbanski, R. Marchany, and J. Tront, "MT6D: A Moving Target IPv6 Defense," in *2011 - MILCOM 2011 Military Communications Conference*, Nov. 2011, pp. 1321–1326.

[119] ——, "The Blind Man's Bluff Approach to Security Using IPv6," *IEEE Security & Privacy*, vol. 10, no. 4, pp. 35–43, Jul. 2012.

[120] P. Kampanakis, H. G. Perros, and T. Beyene, "SDN-based solutions for Moving Target Defense network protection," *Proceeding of IEEE International Symposium on a World of Wireless, Mobile and Multimedia Networks*, 2014.

[121] Z. Zhao, F. Liu, and D. Gong, "An SDN-Based Fingerprint Hopping Method to Prevent Fingerprinting Attacks," *Security and Communication Networks*, 2017.

[122] J. S. Banks and J. Sobel, "Equilibrium Selection in Signaling Games," *Econometrica*, vol. 55, no. 3, p. 647, May 1987.

[123] S. Venkatesan, M. Albanese, G. Cybenko, and S. Jajodia, "A Moving Target Defense Approach to Disrupting Stealthy Botnets," in *Proceedings of the 2016 ACM Workshop on Moving Target Defense*, ser. MTD '16. Vienna, Austria: Association for Computing Machinery, Oct. 2016, pp. 37–46.

[124] N. E. Friedkin, "Theoretical Foundations for Centrality Measures," *American Journal of Sociology*, vol. 96, no. 6, pp. 1478–1504, 1991.

[125] S. Sengupta, A. Chowdhary, D. Huang, and S. Kambhampati, "Moving Target Defense for the Placement of Intrusion Detection Systems in the Cloud," in *Decision and Game Theory for Security*, ser. Lecture Notes in Computer Science, L. Bushnell, R. Poovendran, and T. Başar, Eds. Cham: Springer International Publishing, 2018, pp. 326–345.

[126] D. Korzhyk, V. Conitzer, and R. Parr, "Complexity of Computing Optimal Stackelberg Strategies in Security Resource Allocation Games," *Proceedings of the Twenty-Fourth AAAI Conference on Artificial Intelligence (AAAI-10)*, p. 6, 2010.

[127] C. Sinclair, L. Pierce, and S. Matzner, "An application of machine learning to network intrusion detection," in *Proceedings 15th Annual Computer Security Applications Conference (ACSAC'99)*, Dec. 1999, pp. 371–377.

[128] C. Yin, Y. Zhu, J. Fei, and X. He, "A Deep Learning Approach for Intrusion Detection Using Recurrent Neural Networks," *IEEE Access*, vol. 5, pp. 21 954–21 961, 2017.

[129] W. Wang, Y. Sheng, J. Wang, X. Zeng, X. Ye, Y. Huang, and M. Zhu, "HAST-IDS: Learning Hierarchical Spatial-Temporal Features Using Deep Neural Networks to Improve Intrusion Detection," *IEEE Access*, vol. 6, pp. 1792–1806, 2018.

[130] C.-H. Huang, T.-H. Lee, L.-h. Chang, J.-R. Lin, and G. Horng, "Adversarial Attacks on SDN-Based Deep Learning IDS System," in *Mobile and Wireless Technology 2018*, ser. Lecture Notes in Electrical Engineering, K. J. Kim and H. Kim, Eds. Singapore: Springer, 2018, pp. 181–191.

[131] M. Usama, M. Asim, S. Latif, J. Qadir, and Ala-Al-Fuqaha, "Generative Adversarial Networks For Launching and Thwarting Adversarial

Attacks on Network Intrusion Detection Systems," in *2019 15th International Wireless Communications Mobile Computing Conference (IWCMC)*, Jun. 2019, pp. 78–83.

[132] Z. Lin, Y. Shi, and Z. Xue, "IDSGAN: Generative Adversarial Networks for Attack Generation against Intrusion Detection," *arXiv:1809.02077 [cs]*, Jun. 2019.

[133] Y. Vorobeychik and B. Li, "Optimal randomized classification in adversarial settings," in *Proceedings of the 2014 International Conference on Autonomous Agents and Multi-Agent Systems*, ser. AAMAS '14. Paris, France: International Foundation for Autonomous Agents and Multiagent Systems, May 2014, pp. 485–492.

[134] S. Sengupta, T. Chakraborti, and S. Kambhampati, "MTDeep: Boosting the Security of Deep Neural Nets Against Adversarial Attacks with Moving Target Defense," in *Decision and Game Theory for Security*. Cham: Springer International Publishing, 2019, pp. 479–491.

[135] "PaX Address Space Layout Randomization," https://pax.grsecurity.net/docs/aslr.txt, 2003.

[136] L. Li, J. Just, and R. Sekar, "Address-Space Randomization for Windows Systems," in *2006 22nd Annual Computer Security Applications Conference (ACSAC'06)*. Miami Beach, FL, USA: IEEE, Dec. 2006, pp. 329–338.

[137] G. S. Kc, A. D. Keromytis, and V. Prevelakis, "Countering code-injection attacks with instruction-set randomization," in *Proceedings of the 10th ACM Conference on Computer and Communications Security*, ser. CCS '03. Washington D.C., USA: Association for Computing Machinery, Oct. 2003, pp. 272–280.

[138] M. Thompson, N. Evans, and V. Kisekka, "Multiple OS rotational environment an implemented Moving Target Defense," in *2014 7th International Symposium on Resilient Control Systems (ISRCS)*, Aug. 2014, pp. 1–6.

[139] H. Okhravi, A. Comella, E. Robinson, S. Yannalfo, P. Michaleas, and J. Haines, "Creating a Cyber Moving Target for Critical Infrastructure Applications," in *International Conference on Critical Infrastructure Protection*, ser. IFIP Advances in Information and Communication Technology, Hanover, NH, USA, 2011, pp. 107–123.

[140] P. Larsen, A. Homescu, S. Brunthaler, and M. Franz, "SoK: Automated Software Diversity," in *2014 IEEE Symposium on Security and Privacy*, May 2014, pp. 276–291.

[141] M. Azab, R. Hassan, and M. Eltoweissy, "ChameleonSoft: A moving target defense system," in *7th International Conference on Collaborative Computing: Networking, Applications and Worksharing (CollaborateCom)*, Oct. 2011, pp. 241–250.

[142] A. Gupta, S. Kerr, M. S. Kirkpatrick, and E. Bertino, "Marlin: A Fine Grained Randomization Approach to Defend against ROP Attacks," in *Network and System Security*, ser. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 2013, pp. 293–306.

[143] C. Le Goues, T. Nguyen, S. Forrest, and W. Weimer, "GenProg: A Generic Method for Automatic Software Repair," *IEEE Transactions on Software Engineering*, vol. 38, no. 1, pp. 54–72, Jan. 2012.

[144] E. Germano da Silva, L. A. Dias Knob, J. A. Wickboldt, L. P. Gaspary, L. Z. Granville, and A. Schaeffer-Filho, "Capitalizing on SDN-based SCADA systems: An anti-eavesdropping case-study," in *2015 IFIP/IEEE International Symposium on Integrated Network Management (IM)*, May 2015, pp. 165–173.

[145] A. Aseeri, N. Netjinda, and R. Hewett, "Alleviating eavesdropping attacks in software-defined networking data plane," in *Proceedings of the 12th Annual Conference on Cyber and Information Security Research*, ser. CISRC '17. Oak Ridge, Tennessee, USA: Association for Computing Machinery, Apr. 2017, pp. 1–8.

[146] J. Yackoski, P. Xie, H. Bullen, J. Li, and K. Sun, "A Self-shielding Dynamic Network Architecture," in *Military Communications Conference (MILCOM 2011)*, Nov. 2011, pp. 1381–1386.

[147] J. Yackoski, H. Bullen, X. Yu, and J. Li, "Applying Self-Shielding Dynamics to the Network Architecture," in *Moving Target Defense II*, ser. Advances in Information Security. New York, NY: Springer, 2013, pp. 97–115.

[148] Y. Zhang, A. Juels, M. K. Reiter, and T. Ristenpart, "Cross-VM side channels and their use to extract private keys," in *Proceedings of the 2012 ACM Conference on Computer and Communications Security*, ser. CCS '12. Raleigh, North Carolina, USA: Association for Computing Machinery, Oct. 2012, pp. 305–316.

[149] E. Pattuk, M. Kantarcioglu, Z. Lin, and H. Ulusoy, "Preventing cryptographic key leakage in cloud virtual machines," in *Proceedings of the 23rd USENIX Conference on Security Symposium*, ser. SEC'14. San Diego, CA: USENIX Association, Aug. 2014, pp. 703–718.

[150] A. Beimel, "Secret-Sharing Schemes: A Survey," in *Coding and Cryptology*. Berlin, Heidelberg: Springer, 2011, pp. 11–46.

[151] C. Smutz and A. Stavrou, "Preventing Exploits in Microsoft Office Documents Through Content Randomization," in *RAID*, 2015.

[152] "Deception in Depth – The Case for a Full-Stack Architecture," https://trapx.com/deception-in-depth-the-case-for-a-full-stack-architecture/, Mar. 2017.

[153] W. Wang, Jeffrey Bickford, Ilona Murynets, Ramesh Subbaraman, and Gokul Singaraju, "Detecting Targeted Attacks By Multilayer Deception," *Journal of Cyber Security and Mobility*, vol. 2, no. 2, pp. 175–199, 2013.

[154] J. Sun and K. Sun, "DESIR: Decoy-enhanced seamless IP randomization," in *IEEE INFOCOM 2016 - The 35th Annual IEEE International Conference on Computer Communications*, Apr. 2016, pp. 1–9.

[155] K. Park, S. Woo, D. Moon, and H. Choi, "Secure Cyber Deception Architecture and Decoy Injection to Mitigate the Insider Threat," *Symmetry*, vol. 10, no. 1, p. 14, Jan. 2018.

[156] J. H. Jafarian, A. Niakanlahiji, E. Al-Shaer, and Q. Duan, "Multi-dimensional Host Identity Anonymization for Defeating Skilled Attackers," in *Proceedings of the 2016 ACM Workshop on Moving Target Defense - MTD'16*. Vienna, Austria: ACM Press, 2016, pp. 47–58.

[157] Y. Shi, H. Zhang, J. Wang, F. Xiao, J. Huang, D. Zha, H. Hu, F. Yan, and B. Zhao, "CHAOS: An SDN-Based Moving Target Defense System," *Security and Communication Networks*, 2017.

[158] A. Adebayo and D. B. Rawat, "Deceptor-in-the-Middle (DitM): Cyber Deception for Security in Wireless Network Virtualization," in *2020 IEEE 17th Annual Consumer Communications Networking Conference (CCNC)*, Jan. 2020, pp. 1–6.

[159] K. Jiang and H. Zheng, "Design and Implementation of A Machine Learning Enhanced Web Honeypot System," in *2020 13th International Congress on Image and Signal Processing, BioMedical Engineering and Informatics (CISP-BMEI)*, Oct. 2020, pp. 957–961.

[160] J. Sun, K. Sun, and Q. Li, "CyberMoat: Camouflaging critical server infrastructures with large scale decoy farms," in *2017 IEEE Conference on Communications and Network Security (CNS)*, Oct. 2017, pp. 1–9.

[161] ——, "Towards a Believable Decoy System: Replaying Network Activities from Real System," in *2020 IEEE Conference on Communications and Network Security (CNS)*, Jun. 2020, pp. 1–9.

[162] M. Albanese, E. Battista, and S. Jajodia, "A deception based approach for defeating OS and service fingerprinting," in *2015 IEEE Conference on Communications and Network Security (CNS)*, Sep. 2015, pp. 317–325.

[163] P. Karuna, H. Purohit, S. Jajodia, R. Ganesan, and O. Uzuner, "Fake Document Generation for Cyber Deception by Manipulating Text Comprehensibility," *IEEE Systems Journal*, pp. 1–11, 2020.

[164] J. Sun, S. Liu, and K. Sun, "A Scalable High Fidelity Decoy Framework against Sophisticated Cyber Attacks," in *Proceedings of the 6th ACM Workshop on Moving Target Defense*, ser. MTD'19. New York, NY, USA: Association for Computing Machinery, Nov. 2019, pp. 37–46.

[165] S. T. Trassare, R. Beverly, and D. Alderson, "A Technique for Network Topology Deception," in *MILCOM 2013 - 2013 IEEE Military Communications Conference*, Nov. 2013, pp. 1795–1800.

[166] Q. Duan, E. Al-Shaer, and J. Xie, "Range and Topology Mutation Based Wireless Agility," in *Proceedings of the 7th ACM Workshop on Moving Target Defense*, ser. MTD'20. New York, NY, USA: Association for Computing Machinery, Nov. 2020, pp. 59–67.

[167] J. L. Rrushi, "DNIC Architectural Developments for 0-Knowledge Detection of OPC Malware," *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 1, pp. 30–44, Jan. 2021.

[168] J. Choi, H. Lee, Y. Park, H. K. Kim, J. Lee, Y. Kim, G. Lee, S.-W. Shim, and T. Kim, "PhantomFS-v2: Dare You to Avoid This Trap," *IEEE Access*, vol. 8, pp. 198 285–198 300, Oct. 2020.

[169] C. J. Chiang, Y. M. Gottlieb, Shridatt James Sugrim, R. Chadha, C. Serban, A. Poylisher, L. M. Marvel, and J. Santos, "ACyDS: An adaptive cyber deception system," in *MILCOM 2016 - 2016 IEEE Military Communications Conference*, Nov. 2016, pp. 800–805.

[170] M. H. Almeshekah, "Using Deception to Enhance Security: A Taxonomy, Model, and Novel Uses," Ph.D. dissertation, Jan. 2015.

[171] C. De Faveri, A. Moreira, and V. Amaral, "Goal-Driven Deception Tactics Design," in *2016 IEEE 27th International Symposium on Software Reliability Engineering (ISSRE)*. Ottawa, ON, Canada: IEEE, Oct. 2016, pp. 264–275.

[172] K. E. Heckman, F. J. Stech, B. S. Schmoker, and R. K. Thomas, "Denial and Deception in Cyber Defense," *IEEE Computer*, vol. 48, no. 4, pp. 36–44, Apr. 2015.

[173] C. De Faveri and A. Moreira, "Designing Adaptive Deception Strategies," in *2016 IEEE International Conference on Software Quality, Reliability and Security Companion (QRS-C)*, Aug. 2016, pp. 77–84.

[174] R. Budiarto, A. Samsudin, C. Heong, and S. Noori, "Honeypots: Why we need a dynamics honeypots?" in *Proceedings of International Conference on Information and Communication Technologies: From Theory to Applications*, Apr. 2004, pp. 565–566.

[175] W. Zanoramy Ansiry Zakaria and M. Laiha Mat Kiah, "A review of dynamic and intelligent honeypots," *ScienceAsia*, vol. 39S, 2013.

[176] S. Wang, Q. Pei, J. Wang, G. Tang, Y. Zhang, and X. Liu, "An Intelligent Deployment Policy for Deception Resources Based on Reinforcement Learning," *IEEE Access*, vol. 8, pp. 35 792–35 804, 2020.

[177] G. Wagener, R. State, T. Engel, and A. Dulaunoy, "Adaptive and self-configurable honeypots," in *12th IFIP/IEEE International Symposium on Integrated Network Management (IM 2011) and Workshops*, May 2011, pp. 345–352.

[178] G. Wagener, R. State, A. Dulaunoy, and T. Engel, "Heliza: Talking dirty to the attackers," *Journal in Computer Virology*, vol. 7, no. 3, pp. 221–232, Aug. 2011.

[179] H. S. Baird, A. L. Coates, and R. J. Fateman, "PessimalPrint: A reverse Turing test," *International Journal on Document Analysis and Recognition*, vol. 5, no. 2, pp. 158–163, Apr. 2003.

[180] A. Pauna and I. Bica, "RASSH - Reinforced adaptive SSH honeypot," in *Proceedings of the 10th International Conference on Communications (COMM)*, May 2014, pp. 1–6.

[181] T. E. Carroll and D. Grosu, "A Game Theoretic Investigation of Deception in Network Security," in *Proceedings of 18th International Conference on Computer Communications and Networks*, 2009, p. 6.

[182] M. A. Rahman, M. H. Manshaei, and E. Al-Shaer, "A game-theoretic approach for deceiving Remote Operating System Fingerprinting," in *Proceedings of IEEE Conference on Communications and Network Security (CNS)*, Oct. 2013, pp. 73–81.

[183] K. M. Carter, J. F. Riordan, and H. Okhravi, "A Game Theoretic Approach to Strategy Determination for Dynamic Platform Defenses," in *Proceedings of the First ACM Workshop on Moving Target Defense*, ser. MTD '14. Scottsdale, Arizona, USA: Association for Computing Machinery, Nov. 2014, pp. 21–30.

[184] C. Lei, D.-H. Ma, and H.-Q. Zhang, "Optimal Strategy Selection for Moving Target Defense Based on Markov Game," *IEEE Access*, vol. 5, pp. 156–169, 2017.

[185] K. Horák, Q. Zhu, and B. Bošanský, "Manipulating Adversary's Belief: A Dynamic Game Approach to Deception by Design for Proactive Network Security," in *International Conference on Decision and Game Theory for Security*, vol. 10575, 2017, pp. 273–294.

[186] Q. Zhu and T. Başar, "Game-Theoretic Approach to Feedback-Driven Multi-stage Moving Target Defense," in *Decision and Game Theory for Security*, ser. Lecture Notes in Computer Science, 2013, pp. 246–263.

[187] N. C. Rowe, "A model of deception during cyber-attacks on information systems," in *IEEE First Symposium onMulti-Agent Security and Survivability, 2004*, Aug. 2004, pp. 21–30.

[188] Anton Chuvakin, "Will Deception Fizzle ... Again?" Mar. 2019.

[189] M. M. Islam and E. Al-Shaer, "Active Deception Framework: An Extensible Development Environment for Adaptive Cyber Deception," in *2020 IEEE Secure Development (SecDev)*, Sep. 2020, pp. 41–48.

[190] T. S. Inc, "TrapX Introduces Industry-First Deception-As-A-Service Solution, TrapX Flex™," https://www.prnewswire.com/news-releases/trapx-introduces-industry-first-deception-as-a-service-solution-trapx-flex-301162363.html, Oct. 2020.

[191] D. Ye, T. Zhu, S. Shen, and W. Zhou, "A Differentially Private Game Theoretic Approach for Deceiving Cyber Adversaries," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 569–584, 2021.

[192] J. C. Acosta, A. Basak, C. Kiekintveld, N. Leslie, and C. Kamhoua, "Cybersecurity Deception Experimentation System," in *2020 IEEE Secure Development (SecDev)*, Sep. 2020, pp. 34–40.

[193] K. Ferguson-Walter, T. Shade, A. Rogers, M. C. S. Trumbo, K. S. Nauer, K. M. Divis, A. Jones, A. Combs, and R. G. Abbott, "The Tularosa Study: An Experimental Design and Implementation to Quantify the Effectiveness of Cyber Deception." Sandia National Lab. (SNL-NM), Albuquerque, NM (United States), Tech. Rep. SAND2018-5870C, May 2018.

[194] Q. Duan, E. Al-Shaer, M. Islam, and H. Jafarian, "CONCEAL: A Strategy Composition for Resilient Cyber Deception-Framework, Metrics and Deployment," in *2018 IEEE Conference on Communications and Network Security (CNS)*, May 2018, pp. 1–9.

[195] H. Maleki, S. Valizadeh, W. Koch, A. Bestavros, and M. van Dijk, "Markov Modeling of Moving Target Defense Games," in *Proceedings of the 2016 ACM Workshop on Moving Target Defense - MTD'16*. Vienna, Austria: ACM Press, 2016, pp. 81–92.

[196] N. Abu-Ghazaleh, D. Ponomarev, and D. Evtyushkin, "How the spectre and meltdown hacks really worked," *IEEE Spectrum*, vol. 56, no. 3, pp. 42–49, Mar. 2019.

[197] M. Gallagher, L. Biernacki, S. Chen, Z. B. Aweke, S. F. Yitbarek, M. T. Aga, A. Harris, Z. Xu, B. Kasikci, V. Bertacco, S. Malik, M. Tiwari, and T. Austin, "Morpheus: A Vulnerability-Tolerant Secure Architecture Based on Ensembles of Moving Target Defenses with Churn," in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. Providence RI USA: ACM, Apr. 2019, pp. 469–484.

[198] K. J. Ferguson-Walter, "An Empirical Assessment of the Effectiveness of Deception for Cyber Defense," Ph.D. dissertation, University of Massachusetts Amherst, Feb. 2020.